GH(M)3x2x 驱动库移植指南

V 4.4

汇顶机密
GoodiX Confidential For pn@lierda.com Only

====== 免责声明======

本出版物中所述的器件应用信息及其他类似内容仅为您提供便利，它们可能由更新之信息所替代。确保应用符合技术规范，是您自身应负的责任。深圳市汇顶科技股份有限公司（以下简称“GOODIX”）对这些信息不作任何明示或暗示、书面或口头、法定或其他形式的声明或担保，包括但不限于针对其使用情况、质量、性能、适销性或特定用途的适用性的声明或担保。GOODIX 对因这些信息及使用这些信息而引起的后果不承担任何责任。未经GOODIX书面批准，不得将GOODIX 的产品用作生命维持系统中的关键组件。在GOODIX 知识产权保护下，不得暗中或以其他方式转让任何许可证。

目录

1 概述 6

1.1 本文档目的 6

1.2 假定 6

1.3 要求 6

1.3.1 硬件运行环境 6

1.3.2 软件运行环境 6

1.4 特性 6

1.5 示例和演示 7

1.6 数据类型 7

1.7 驱动库文件结构 7

2 驱动库功能简介 9

2.1 驱动库文件介绍 9

3 GH(M)3X2X_DEMO 例程移植步骤 9

3.1 应用场景 10

3.2 例程文件结构及对外接口 10

3.3 将驱动库例程添加进工程 11

3.4 将算法调用例程添加进工程 12

3.5 修改配置文件 12

3.5.1 BSP 配置 12

3.5.2 APP 配置 15

3.5.3 PPG 功能配置 16

3.5.4 调光相关配置 17

3.5.5 Lead 配置 17

3.5.6 调试类配置 18

3.5.7 手动切换配置功能 20

3.5.8 其他配置 21

3.6 添加用户代码 21

3.6.1 Log 函数 22

3.6.2 芯片读写单元操作函数 22

3.6.3 延时函数 25

3.6.4 调光信息处理 25

3.6.5 动态内存申请和释放函数 25

汇顶机密
Goodix Confidential For pn@lierda.com Only

3.6.6 其他用户代码函数 26
3.7 添加 G-SENSOR(ACC+GYRO)相关接口 26
3.7.1 相关接口 26
3.7.2 同步读取 26
3.7.3 异步读取 27
3.7.4 时间戳对齐 27
3.8 添加驱动库配置表 27
3.9 驱动和算法分开运行的修改 31
3.10 初始化接口调用 31
3.11 采样控制 31
3.11.1 开始和停止 31
3.11.2 采样率的预设定 31
3.11.3 采样通道使能的预设定 31
3.11.4 Fifo 中断水线设定 32
3.12 GH3x2x 中断处理 32
3.12.1 中断模式动态切换 33
3.13 外层接口调用流程的设计和编写 33
3.13.1 带操作系统的轮询模式 33
3.13.2 不带操作系统的轮询模式 34
3.13.3 带操作系统的中断模式 35
3.13.4 不带操作系统的中断模式 36
3.14 佩戴检测定时器处理 37
3.15 应用模式切换 38
3.16 驱动 HOOK 功能 38
3.17 工程模式 39
3.17.1 应用示例 39
3.17.2 注意 40
3.18 DEMO CODE 文件修改 40
3.19 线程布置 40
3.20 其他 40
4 佩戴检测模块移植 41
4.1 硬件佩戴检测移植步骤 41
4.1.1 修改宏配置 41
4.2 软件佩戴检测流程图说明 41
4.3 软件佩戴检测移植步骤 42
4.3.1 修改宏配置 42

汇顶机密
Goodix Confidential For pn@lieda.com Only

4.3.2 添加用户代码以及增加接口调用 42
4.4 活体检测函数应用示例 43
5 ECG 功能使用说明 44
5.1 推荐的控制流程 44
5.2 算法结果获取和处理 45
6 GOODIX 通信协议移植 47
6.1 移植步骤 47
6.1.1 开启协议总开关 47
6.1.2 选择协议类型 47
6.1.3 调整协议相关宏配置 47
6.1.4 BLE 模块设置 48
6.1.5 下行接口对接（可选） 48
6.1.6 上行接口对接 48
7 六导联 ECG 功能的额外的移植步骤 50
7.1 相关宏配置 50
7.2 I2C 接口注册 50
7.3 SPI 接口注册 50
7.4 配置表存放 51
7.5 函数的推荐调用流程 51
7.6 算法结果获取和处理 52
8 驱动和算法分开运行详细修改指引 52
8.1 驱动和算法交互机制 53
8.1.1 常规方式的交互机制 53
8.1.2 分开运行方式的交互机制 53
8.2 修改步骤 55
8.2.1 驱动层的文件修改和编译 55
8.2.2 算法层的文件修改和编译 55
8.2.3 g_pstGh3x2xFrameInfo 的填充 56
9 附录一：DEMO 与驱动库函数查询表 56
10 附录二：STGH3X2XFRAMEINFO 的定义 66
11 附录三：STGH3X2XENGINEERINGMODESAMPLEPARAM 定义 68
12 附录四：移植过程常见问题排查 70
12.1 编译出错 70

汇顶机密
Goodix Confidential For Pn@lieda.com only

12.2 链接出错 70
12.3 运行异常 71
13 附录四：I2C 设备地址宏配置与 PIN 脚连接关系 72
14 版本记录 73

汇顶机密
Goodix Confidential For pn@lierda.com Only

1 概述

本章主要介绍本文档的写作目的与背景，以及驱动库的一些特性。

对应驱动版本号: v4.0.1.2

1.1 本文档目的

本文档用来介绍如何移植 GH(M)3x2x 的驱动库，还将说明 GH(M)3x2x 驱动库的内部结构，以及所具有的功能。（以下简称为驱动库）

1.2 假定

本文档假定您已经具备 C 程序设计语言的基础知识，对 GH(M)3x2x 芯片的功能有初步的了解。

1.3 要求

1.3.1 硬件运行环境

一般运行环境为 ARM M4 等 32bits MCU 平台，需要带 I2C 或者 SPI 接口。

1.3.2 软件运行环境

需要 C 标准库 <stdio.h>、 <stdlib.h>、 <string.h> 支持（可能需要依赖 snprintf 函数，详见添加用户代码->Log 函数一节）。

本驱动库暂不支持生成动态链接库，例如安卓的 so 文件或 windos 的 DLL 文件。

1.4 特性

驱动库为用户提供单驱动接口。单驱动接口不包含算法相关的操作，只和 GH(M)3x2x 芯片基本功能相关；带算法的驱动接口，会针对不同的算法，封装对应的开启和停止采样、算法 rawdata 打包等类型的接口，用户无需关注驱动库内部操作原理和细节，操作起来会更简单。

算法调用请参考《Goodix 算法调用 SDK 接口说明书.docx》。

此外，驱动库还具有以下特性：

- 数据调试方便。可实现用户 MCU 和 Goodix 上位机(EVK/APP)之间的通信，供用户开发过程中对采样数据进行调试；

汇顶科技 机密信息
未经允许 不得转载

问题反馈

- 功能多样。封装了大量芯片功能相关的接口，如软件调光、ADT 处理等，用户可直接使用。
- 功能可裁剪。
- 可移植性。提供 demo 代码，方便用户快速移植。

1.5 示例和演示

GH(M)3x2x_Demo 例程例程为用户封装好了顶层接口，如初始化、中断处理、开启和停止采样等。同时，该例程将需要用户实现的部分独立成一个“.c”文件，并独立出一个“.h”配置文件，用户可通过修改配置文件中的宏定义，来打开或者关闭某一功能，无需改动其他部分的代码。

1.6 数据类型

由于 C 语言不提供在其他所有平台上都相同的固定长度的数据类型，因此大多数情况下，驱动库使用重定义的数据类型，如下表所示：

表 1-1 驱动库数据类型

数据类型	描述
GU8	无符号的 8 位值
GS8	有符号的 8 位值
GU16	无符号的 16 位值
GS16	有符号的 16 位值
GU32	无符号的 32 位值
GS32	有符号的 32 位值
GF32	32 位浮点值
GD32	64 位浮点值
GCHAR	字符
GBOOL	布尔值
GS64	有符号的 64 位值

1.7 驱动库文件结构

用户可使用的驱动库文件包含如下部分：

汇顶科技 机密信息
未经允许 不得转载

问题反馈

图 1-1 驱动与 Demo 代码文件列表

“1-drv_lib”文件夹中包含编译出来的驱动库文件，分为两个版本，一种是 release 版本，一种是 debug 版本，debug 版本可以输出驱动库打印信息（用户要实现打印函数）。用户可根据自身平台特性，选择合适的驱动库。

图 1-2 驱动库输出版本

“2-demo_code”文件夹中包含三个模块的例程，包括：

1. 算法库调用 SDK，具体用法参考《Goodix 算法调用 SDK 接口说明书》
2. 驱动调用 demo 例程，用户可使用驱动库 demo 例程进行项目开发，可参考单驱动例程理解接口调用规则。
3. 产测项功能 SDK，具体用法参考《GH(M)3x2x 量产测试嵌入式平台开发库移植指南》

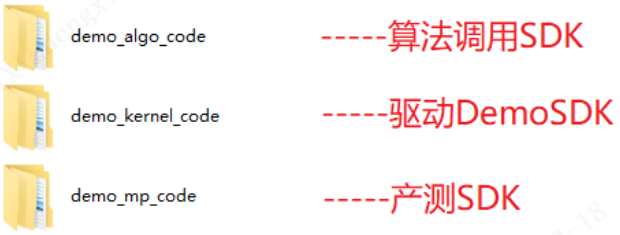


图 1-3 驱动库例程

“4-afe_ref_code”文件夹中包含 AFE 代码例程，具体用法参考《GH(M)3x2x AFE 驱动移植指南》

2 驱动库功能简介

本章节对驱动库的功能做下简单介绍，使用户对驱动库的功能有个整体的了解。

2.1 驱动库文件介绍

GH(M)3X2X 驱动库包含 gh3x2x_drv.h 头文件与 4 个驱动库文件，除 kernel 库外，其余库为可选库。

库文件名	描述
*kernel.a/lib	核心库，它是操作芯片的基础库
*agc.a/lib	调光库，用于动态调整采样参数（如驱动电流、增益）的库
*ecg.a/lib	Ecg 库，用于识别 ecg lead 检测的库
*zip.a/lib	通信协议库，该库包括了协议数据的封装，协议数据的压缩，如果需要用 goodix app 连接驱动库，需要使用该库

3 GH(M)3X2X_Demo 例程移植步骤

本章节介绍 GH(M)3x2x 例程的移植步骤。用户可按照该步骤，将例程代码移植到用户应用程序中。**移植后如果出现问题优先查看附录“移植过程常见问题排查”。**

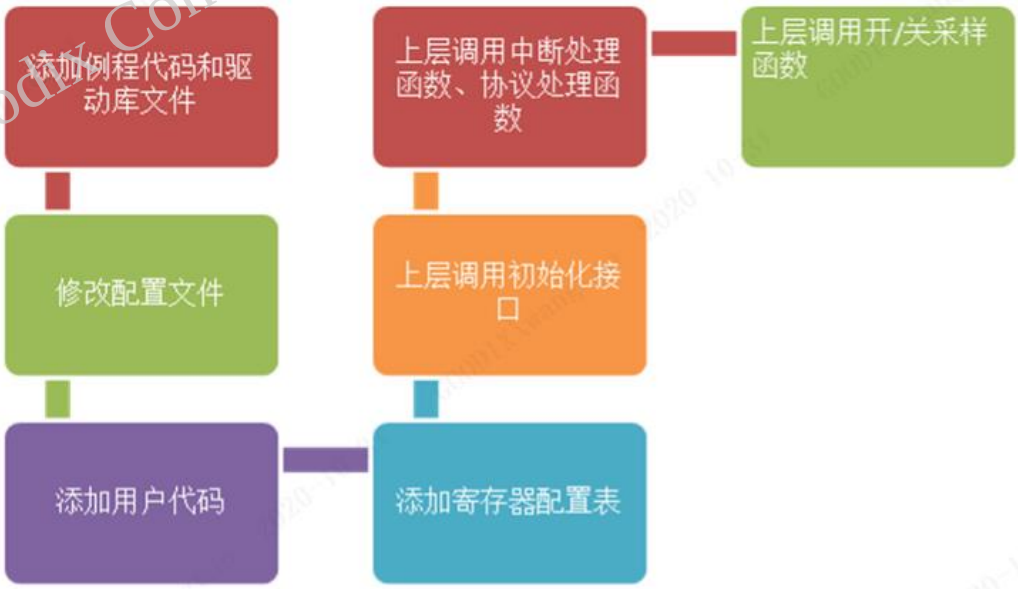


图 3-1 驱动库移植流程

3.1 应用场景

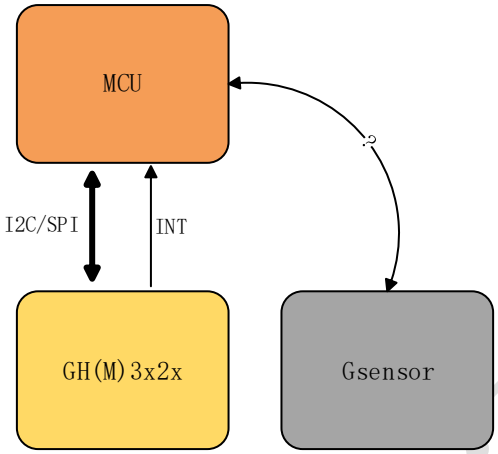


图 3-2 驱动库例程应用场景

GH(M)3X2X_Demo 例程的应用场景如上图所示，MCU 和 GH(M)3x2x 之间通过 I2C/SPI 进行通信，同时，GH(M)3x2x 和 MCU 之间还有中断管脚连接。

用户如需使用 G-sensor 数据，还需增加 G-sensor 芯片和 MCU 之间的连接。MCU 和 G-sensor 之间的交互由客户自行设计，只需要保证 G-sensor 模块提供的采样数据，和 GH(M)3x2x 的采样数据处于同一时间段内即可。

3.2 例程文件结构及对外接口

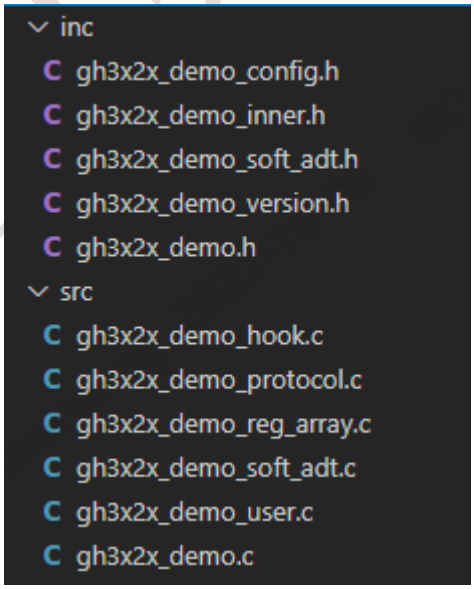


图 3-3 例程文件结构

上图为本例程的目录结构。其中，“inc”文件夹包含“gh3x2x_demo.h”等头文件：

汇顶机密
Goodix Confidential For Internal Use Only

头文件中已封装好了用户必要的顶层接口，原则上，用户无需调用其他文件中的接口，即可完成初始化、采样处理、中断处理等一系列操作。

“src/inc”文件夹中包含例程源码：

表 3-1 源码文件

文件	描述
gh3x2x_demo.c	包含初始化、中断处理等流程相关的代码
gh3x2x_demo_hook.c	驱动库关键流程的 hook 函数
gh3x2x_demo_protocol.c	驱动库协议处理相关的函数
gh3x2x_demo_reg_array.c	驱动库的寄存器配置表文件
gh3x2x_demo_user.c	需要用户实现的平台相关的代码
gh3x2x_demo_soft_adt.c	活体检测流程
gh3x2x_demo_config.h	用户可进行配置的头文件

上表中，高亮部分是用户需要根据自身应用需要手动更改的文件。

“gh3x2x_demo.h”头文件中，对外提供的接口见附录一。

驱动库的每种应用模式都会有对应的驱动库寄存器配置表，切换应用模式，本质上是切换到包含该应用功能的配置表。如果用户使用了多种配置表，可调用该函数在多张配置表之间进行切换。详细描述可参考添加驱动库配置表章节和驱动库模式切换章节。

如果用户添加的每张配置表中，都只包含 HR/ECG/SPO2 中的一种采样功能（（ADT 可有可无）），比如 HR+ADT、SPO2+ADT、ECG+ADT，则无需调用该函数，例程中会根据开启采样的功能，自动切换配置表。

前面的两节介绍了驱动库例程的文件和接口，以下章节将介绍如何将驱动库例程移植进用户的工程中，并根据应用场景对驱动库做配置。

3.3 将驱动库例程添加进工程

驱动库对外提供“.h”头文件和“.lib”库文件，用户可根据产品使用的平台，选择相应的驱动库。

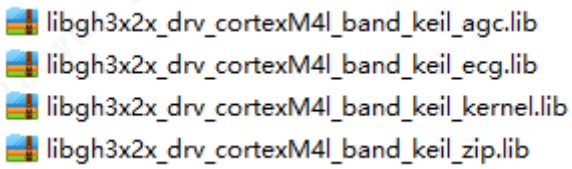


图 3-4 驱动库文件(以 Keil 平台为例)

将“gh3x2x.h”和选择的“.lib”文件一同拷贝到例程的“demo_kernel_code\src”目录中：

图 3-5 添加驱动库文件(以 keil 平台为例)

将例程中的代码和库文件添加进用户工程中，头文件包含到例程的“inc”和“src”路径。建议用户将驱动库例程相关的代码和用户程序代码分开存放，这样可以方便后续驱动库版本的更新。

3.4 将算法调用例程添加进工程

请参考文档《Goodix 算法调用 SDK 接口说明书.docx》。

3.5 修改配置文件

本节讲解驱动库例程的配置方式，用户可根据需求，对“gh3x2x_demo_config.h”文件中的宏定义进行修改。可配置的参数如下：

3.5.1 BSP 配置

表 3-2 BSP 配置表

宏	可配置值	描述
__GH3X2X_INTERFACE__	__GH3X2X_INTERFACE_I2C__	使用 I2C 接口通信
	__GH3X2X_INTERFACE_SPI__	使用 SPI 接口通信 **仅支持极性为 0，相位为 0 的模式的 SPI**
#if(__GH3X2X_INTERFACE__ == __GH3X2X_INTERFACE_SPI__)		

__GH3X2X_SPI_TYPE__	__GH3X2X_SPI_TYPE_SOFTWARE_CS__	驱动内操作 CS 引脚
	__GH3X2X_SPI_TYPE_HARDWARE_CS__	CS 引脚由外部操作
#endif		
#if(__GH3X2X_INTERFACE__ == __GH3X2X_INTERFACE_I2C__)		
__GH3X2X_I2C_DEVICE_ID__	默认值: GH3X2X_I2C_ID_SEL_1L0L	GH(M)3x2x 芯片的 I2C 地址选择管脚电平，请依据硬件原理图选择对应的配置宏。具体参考附录“I2C 设备地址宏配置与 pin 脚连接关系”
__GH3X2X_I2C_DEVICE_SLAVER_ID__	默认值： GH3X2X_I2C_ID_SEL_1H0L	GH(M)3x2x 从芯片的 I2C 地址选择管脚电平（针对级联应用），请依据硬件原理图选择对应的配置宏。具体参考附录“I2C 设备地址宏配置与 pin 脚连接关系”
#endif		
__GS_SENSITIVITY_CONFIG__	参考 GSENSOR_SENSITIVITY_512_COUNTS_PER_G	G-sensor 采样精度 1g 等于多少 LSB
__GS_GYRO_ENABLE__	0/1	六轴传感器
__GS_NONSYNC_READ_EN__	1	允许 GSENSOR 与 GH(M)3X2X PPG 进行异步采样
	0	不允许 GSENSOR 与 GH(M)3X2X PPG 进行异步采样
#if __GS_NONSYNC_READ_EN__		

汇顶科技 机密信息
未经允许 不得转载

问题反馈

__GS_SAMPLE_RATE_HZ__	≥0	用户设置 GSENSOR 采样率
__GS_READ_POINT_NUM__	≥0	GSENSOR 每秒读到点数
__GS_READ_POINT_NUM_JITTER__	一般设置为 3	GS Buffer 大小定位用户读取一次读取的点数再+3，因为一般 Gsensor 读取点数会上下浮动 1~2
#endif		
__GS_TIMESTAMP_READ_EN__	0/1	GSensor 用时间戳对齐
#if __GS_NONSYNC_READ_EN__		
__SYNC_ACC_SAMPLERATE__	≥0	ACC 采样频率
__SYNC_PPG_SAMPLERATE__	≥0	PPG 采样频率
__SYNC_ACC_BUFFER_SECOND__	一般设置为 3	缓存 ACC 数据秒数 一般为 PPG buffer 的 2~3 倍
__SYNC_PPG_BUFFER_SECOND__	一般设置为 1	缓存 PPG 数据秒数
#endif		
__CAP_ENABLE__	0/1	电容 sensor 使能
__TEMP_ENABLE__	0/1	NTC 温度 sensor 使能
__SUPPORT_HARD_RESET_CONFIG__	0：软中断 1：硬中断	软中断为发送 chip cmd 给芯片进行 resetet 硬中断为拉 chip 的 reset 引脚
__INTERRUPT_PROCESS_BY_POLLING__	__NORMAL_INT_PROCESS_MODE__	通过中断方式处理中断事件
	__POLLING_INT_PROCESS_MODE__	通过轮询方式处理中断事件
	__MIX_INT_PROCESS_MODE__	同时支持两种模式，可切换
__PLATFORM_WITHOUT_OS__	1	不使用 OS

汇顶科技机密
Goodix Confidential For pm@lieda.com Only

汇顶科技 机密信息
未经允许 不得转载

	0	使用 OS，如 RTOS
__PASS_THROUGH_MODE__	1	使用透传模式
	0	不使用透传模式
__GH3X2X_MP_MODE__	1	链接产测库
	0	不链接产测库
__GH3X2X_CASCADE_EN__	0	芯片级联使能
__GH3X2X_ECG_RTYPE_5M__	1	RTYPE 为 5m 使能
__RESET_AGCINFO_BEFORE_SAMPLE__	0	初始化时需不需要初始化当前配置的电流/Gain 值
__CURRENT_ECODE_CALIBRATE__	0	根据 efuse 里的校准值，进行电流校准

上述配置中，对于“__PROTOCOL_SERIAL_TYPE__”，如果用户不使用协议解析功能（__SUPPORT_PROTOCOL_ANALYZE__设为 0），则该宏定义无效。

3.5.2 APP 配置

表 3-3 应用类配置表

宏	可配置值	描述
__USER_DYNAMIC_DRV_BUF_EN__	0/1	是否使用动态内存做为驱动库 rawdata buffer（需要用户实现 Gh3x2xMallocUser 以及 Gh3x2xFreeUser）
__DRIVER_LIB_MODE__	__DRV_LIB_WITH_ALGO__	支持算法
	__DRV_LIB_WITHOUT_ALGO__	不支持算法
__FUNC_RUN_SIMULTANEOUSLY_SUPPORT__	1	支持多功能同时运行
	0	只支持单功能运行

Goodix Confidential For pn@ierda.com Only

3.5.3 PPG 功能配置

表 3-4 功能类配置表

宏	可配置值	描述
__FUNC_TYPE_HR_ENABLE__	1	支持 HR 功能
	0	不支持 HR 功能
__FUNC_TYPE_HRV_ENABLE__	1	支持 HRV 功能
	0	不支持 HRV 功能
__FUNC_TYPE_HSM_ENABLE__	1	支持 SPO2 功能
	0	不支持 SPO2 功能
__FUNC_TYPE_SPO2_ENABLE__	1	支持 ECG 功能
	0	不支持 ECG 功能
__FUNC_TYPE_BT_ENABLE__	1	支持 BT 功能
__FUNC_TYPE_RESP_ENABLE__	1	支持 RESP 功能
	0	不支持 RESP 功能
__FUNC_TYPE_AF_ENABLE__	1	支持 AF 功能
	0	不支持 AF 功能
__FUNC_TYPE_TEST_ENABLE__	1	支持 TEST 功能
	0	不支持 TEST 功能
__SUPPORT_HARD_ADT_CONFIG__	1	支持佩戴检测功能
	0	不支持佩戴检测功能
__SUPPORT_SOFT_AGC_CONFIG__	1	打开软件调光功能
	0	关闭软件调光功能
#if (__SUPPORT_HARD_ADT_CONFIG__ && __FUNC_TYPE_HR_ENABLE__)		
__FUNC_TYPE_SOFT_ADT_ENABLE__	1	打开软件佩戴检测功能
	0	关闭软件佩戴检测功能

Goodix Confidential For pn@liedia.com only
汇顶机密

#endif		
__FUNC_TYPE_ECG_ENABLE__	1	支持 ECG 功能
	0	不支持 ECG 功能
#if (__FUNC_TYPE_ECG_ENABLE__)		
__FUNC_TYPE_PWTT_ENABLE__	1	支持 PWTT 功能
	0	不支持 PWTT 功能
__FUNC_TYPE_BP_ENABLE__	1	支持 BP 功能
	0	不支持 BP 功能
__SUPPORT_ECG_LEAD_OFF_DET_800HZ__	1	支持 ECG 采样 800Hz
	0	不支持
#endif		

3.5.4 调光相关配置

表 3-5 调试类配置表

宏	可配置值	描述
GH3X2X_NEW_AGC_SLOT_NUM_LIMIT	0~8	需要调光的 slot 总数
GH3X2X_NEW_AGC_SUB_CHNL_NUM_LIMIT	0~24	需要调光的副通道数（非硬件调光通道）

说明：单 PD 项目 GH3X2X_NEW_AGC_SUB_CHNL_NUM_LIMIT 可以配为 0，以节省 ram 资源。多 PD 项目，会用到多路 RX，需要根据用到的最大通道来做适配。

3.5.5 Lead 配置

表 3-6 调试类配置表

宏	可配置值	描述

Goodix Confidential For pn@lierda.com only
汇顶机密

LEADOFF_DET_IQ_AMP_THR_uV	1500(推荐值)	被判断为 lead off 的 IQ 幅值的绝对值阈值
LEADOFF_DET_IQ_AMP_DIFF_THR_uV	1000(推荐值)	被判断为 lead off 的 IQ 幅值的差分值阈值
LEADOFF_DET_IQ_VAL_THR_mV	80(推荐值)	被判断为 lead off 的电压值阈值

3.5.6 调试类配置

表 3-7 调试类配置表

宏	可配置值	描述
__SUPPORT_PROTOCOL_ANALYZE__	1	支持协议解析和打包功能
	0	不支持协议解析和打包功能
#if (__SUPPORT_PROTOCOL_ANALYZE__)		
__UPLOAD_ALGO_RESULT__	1	不上传算法结果
	0	上传算法结果
__PROTOCOL_SERIAL_TYPE__	__PROTOCOL_SERIAL_USE_UART__	通讯方式为串口
	__PROTOCOL_SERIAL_USE_BLE__	通讯方式为蓝牙
__SUPPORT_ZIP_PROTOCOL__	1	不上传算法结果

汇顶机密 Goodix Confidential For pn@lierda.com Only

	0	上传算法结果
__FIFO_PACKAGE_SEND_ENABLE__	1	不上传算法结果
	0	上传算法结果
__GH3X2X_PROTOCOL_SEND_TIMER_PERIOD__	10	Fifo 发包时间间隔（ms）
__GH3X2X_PROTOCOL_DATA_FIFO_LEN__	64	Fifo 长度
__GH3X2X_PROTOCOL_EVENT_FIFO_LEN__	16	事件 Fifo 长度
__GH3X2X_PROTOCOL_EVENT_WAITING_ACK_TIME__	500	协议等待 ACK 时间
__GH3X2X_PROTOCOL_EVENT_RESEND_NUM__	255	未接收到 ACK 时持续上报次数（255 为持续上报）
__GH3X2X_PROTOCOL_DATA_FUNCTION_INTERCEPT__	0	关闭功能上报数据，默认为 0（全部上报）
#endif		
__SUPPORT_SAMPLE_DEBUG_MODE__	1	打开采样调试功能
	0	关闭采样调试功能
__SUPPORT_ELECTRODE_WEAR_STATUS_DUMP__	1	开启活体检测数据 dump

汇顶机密 Goodix Confidential For pn@lierda.com Only

	0	关闭
__SUPPORT_SAMPLE_DEBUG_MODE__	1	打开采样调试功能
	0	关闭采样调试功能
__EXAMPLE_LOG_ TYPE __	__EXAMPLE_LOG_DISABLE__	关闭 log 功能
	__EXAMPLE_LOG_METHOD_0__	启用 log 方式 1
	__EXAMPLE_LOG_METHOD_1__	启用 log 方式 2
__EXAMPLE_LOG_LEVEL__	__EXAMPLE_LOG_LV_0__	无 log
	__EXAMPLE_LOG_LV_1__	打开主要 log
	__EXAMPLE_LOG_LV_2__	打开全部 log
EXAMPLE_LOG_MAX_LEN	≥0	LOG 最大长度
__SUPPORT_ENGINEERING_MODE__	1	支持工程模式
	0	不支持工程模式
__GH3X2X_MEM_POOL_CHECK_EN__	1	支持内存池检测
	0	不支持

注意：采样调试功能（sample debug）不建议用户使用

3.5.7 手动切换配置功能

表 3-8 手动切换配置功能

宏	可配置值	描述

汇顶科技 机密信息
未经允许 不得转载

问题反馈

__SUPPORT_FUNCTION_SAMPLE_RATE_MODIFY__	1	支持动态切换功能采样率
	0	不支持

3.5.8 其他配置

表 3-9 其他配置表

宏	可配置值	描述
__GH3X2X_CFG_LIST_MAX_NUM__	1 - 8	驱动库寄存器配置表的最大个数
__SUPPORT_HOOK_FUNC_CONFIG__	1	支持驱动库初始化、开启采样、停止采样、读取 FIFO 数据的 hook 功能
	0	不支持 hook 功能
__SUPPORT_ALGO_INPUT_OUTPUT_DATA_HOOK_CONFIG__	0 或 1	使能该配置。可以从该 Hook 函数中获取算法的输入输出信息
__GH3X2X_RAWDATA_BUFFER_SIZE__	默认值 300*4（单位：byte）	最大可缓存采样数据字节数
__GH3X2X_RAWDATA_SLAVE_BUFFER_SIZE__	默认值 300*4（单位：byte）	从片可缓存采样数据字节数
__GSENSOR_DATA_BUFFER_SIZE__	默认值 500(单位：6 byte)	最大可缓存 500 个 G-sensor 采样数据
__CAP_DATA_BUFFER_SIZE__	200	电容数据 buffer
__TEMP_DATA_BUFFER_SIZE__	200	温度传感器数据 buffer

3.6 添加用户代码

用户需要添加的代码存放在“gh3x2x_demo_user.c”文件中，该部分接口分为以下几类：

汇顶科技 机密信息
未经允许 不得转载

问题反馈

3.6.1 Log 函数

未实现驱动库的快速移植和问题定位，建议用户首先将 log 功能调通。Log 功能需要将 __EXAMPLE_LOG_CONFIG__ 配置成__EXAMPLE_LOG_METHOD_0__ 或 __EXAMPLE_LOG_METHOD_1__，并实现以下对应的函数：

a) __EXAMPLE_LOG_METHOD_0__ 相关函数

表 3-10 snpintf log 函数

函数名	GH3X2X_Log	
描述	log 打印函数	
入参	GCHAR *log_string	已经格式化的字符串，只需用类似 printf 函数打印 如：printf("%s",log_string)
出参	无	
返回值	无	

注意：该方式使用了标准 C 函数 snpintf，请检查用户平台是否支持该函数。如果不支持，建议改用__EXAMPLE_LOG_METHOD_1__，否则可能会出现编译或链接报错。

b) __EXAMPLE_LOG_METHOD_1__ 相关函数

表 3-11 printf log 函数

函数名	GH3X2X_RegisterPrintf	
描述	log 打印函数	
入参	pPrintfUser	指向函数的指针的指针 用户需要此指针的实体赋值。 如：(*pPrintfUser) = printf
出参	无	
返回值	无	

注意：赋值的函数的类型必选是标准 C printf 完全一样的类型，即 int (XXX)(const char *format, ...)，否则可能出现无法正常打印 log 的问题。

3.6.2 芯片读写单元操作函数

如果 MCU 和 GH(M)3x2x 之间通过 I2C 接口通信，用户需要将“__GH3X2X_INTERFACE__”配置为“__GH3X2X_INTERFACE_I2C__”，并需要根据硬件环境，配置“__GH3X2X_I2C_DEVICE_ID__”的值，该值为 I2C 地址低两位的电平值。此外，用户需要完成以下接口的实现。

表 3-12 I2C 操作函数

函数名	hal_gh3x2x_i2c_init

汇顶科技 机密信息
未经允许 不得转载

问题反馈

描述	I2C 接口初始化	
入参	无	
出参	无	
返回值	无	
函数名	**hal_gh3x2x_i2c_write**	
描述	I2C 写数据接口	
入参	GU8 device_id	i2c 地址
	const GU8 write_buffer[]	数据 buffer
	GU16 length	写数据长度（单位：byte）
出参	无	
返回值	错误码 0：成功	
函数名	**hal_gh3x2x_i2c_read**	
描述	I2C 写数据接口	
入参	GU8 device_id	i2c 地址
	const GU8 write_buffer[]	写数据 buffer
	GU16 write_length	写数据长度（单位：byte）
	GU8 read_buffer[]	读数据 buffer
	GU16 read_length	读数据长度（单位：byte）
出参	无	
返回值	错误码 0：成功	

如果 MCU 和 GH(M)3x2x 之间通过 SPI 接口通信，用户需要将“__GH3X2X_INTERFACE__”配置为“__GH3X2X_INTERFACE_SPI__”，并完成以下接口的实现。

表 3-13 SPI 操作函数

函数名	**hal_gh3x2x_spi_init**	
描述	SPI 接口初始化	
入参	无	
出参	无	
返回值	无	
函数名	**hal_gh3x2x_spi_write**	
描述	SPI 写数据接口（注意：该函数接口中，须确保不能包含 SPI 总线 CS 管脚的控制，该管脚仅受驱动库控制。）	
入参	const GU8 write_buffer[]	数据 buffer
	GU16 length	写数据长度（单位：byte）
出参	无	
返回值	错误码 0：成功	
函数名	**hal_gh3x2x_spi_read**	
描述	SPI 读数据接口（注意：该函数接口中，用户无需实现写数据功能，且须确保不能包含 SPI 总线 CS 管脚的控制，该管脚仅受驱动库控制。）	
入参	GU8 read_buffer[]	读数据 buffer
	GU16 length	读数据长度（单位：byte）

汇顶科技 机密信息
未经允许 不得转载

问题反馈

出参	无	
返回值	错误码 0：成功	
函数名	**hal_gh3x2x_spi_cs_ctrl**	
描述	SPI CS 引脚操作接口（软件 CS 使能时需实现该函数）	
入参	GU8 cs_pin_level	cs 引脚高低电平设置
出参	无	
返回值	无	

下面是硬件 CS 操作时需要实现的 spi 操作函数

函数名	**hal_gh3x2x_spi_write_F1_and_read**	
描述	SPI 读数据接口，CS pin 由 SPI 硬件模块自动控制，该函数需实现 CS LOW + F1(W) + read_buf[0](R) + read_buf[1](R) + ... + CS HIGH 时序	
入参	GU8 read_buffer[]	读数据 buffer
	GU16 length	读数据长度（单位：byte）
出参	无	
返回值	错误码 0：成功	

如果 MCU 可以通过硬件复位管脚操作 GH(M)3x2x，用户需要实现该复位管脚的配置函数（请将“__SUPPORT_HARD_RESET_CONFIG__”配置为 1）：

表 3-14 reset 操作函数

函数名	**hal_gh3x2x_reset_pin_init**	
描述	硬件复位管脚初始化	
入参	无	
出参	无	
返回值	无	
函数名	**hal_gh3x2x_reset_pin_ctrl**	
描述	复位管脚电平控制函数	
入参	GU8 pin_level	reset 引脚高低电平设置
出参	无	
返回值	无	

用户需要实现 GH(M)3x2 中断管脚的配置：

表 3-15 INT 操作函数

函数名	**hal_gh3x2x_int_init**
描述	中断管脚初始化
入参	无
出参	无
返回值	无

该函数中，用户需要初始化中断管脚，配置中断触发方式为上升沿触发，并使能该管脚的中断。

汇顶科技 机密信息
未经允许 不得转载

问题反馈

3.6.3 延时函数

表 3-16 延时操作函数

函数名	Gh3x2x_BspDelayUs	
描述	微秒延时函数	
入参	GU16 usUsec	微秒
出参	无	
返回值	无	
函数名	**Gh3x2x_BspDelayMs**	
描述	毫秒延时函数	
入参	GU16 usMsec	毫秒
出参	无	
返回值	无	

3.6.4 调光信息处理

表 3-17 电流处理函数

函数名	Gh3x2x_UserHandleCurrentInfo
描述	用户处理电流信息
入参	无
出参	无
返回值	无

用户可以在此函数中增加读写电流的信息，可以利用函数有 GH3X2X_GetSlotLedCurrent、GH3X2X_SlotLedCurrentConfig。注意：在自动调光的配置下，无法成功写入电流值；

3.6.5 动态内存申请和释放函数

用户使能__USER_DYNAMIC_DRV_BUF_EN__ 或 __USER_DYNAMIC_ALGO_MEM_EN__，则需要实现动态内存申请和释放函数。

表 3-18 内存处理函数

函数名	Gh3x2xMallocUser	
描述	复位管脚电平控制函数	
入参	GU32 unMemSize	内存池大小
出参	无	
返回值	内存地址	
函数名	**Gh3x2xFreeUser**	
描述	复位管脚电平控制函数	
入参	void* MemAddr	内存池地址
出参	无	

汇顶机密
Goodix Confidential For pn@ierda.com Only

返回值	无

3.6.6 其他用户代码函数

其他用户代码隶属于相对较独立的功能模块

表 3-19 其它用户代码函数

功能模块	相关用户函数
佩戴检测	Gh3x2x_CreateAdtConfirmTimer Gh3x2x_StartAdtConfirmTimer Gh3x2x_StopAdtConfirmTimer
通信协议	Gh3x2x_HalSerialSendData Gh3x2xSerialSendTimerInit Gh3x2xSerialSendTimerStop Gh3x2xSerialSendTimerStart

3.7 添加 G-sensor(ACC+GYRO)相关接口

3.7.1 相关接口

本例程不包含 G-sensor 芯片的初始化，以及 MCU 和 G-sensor 之间的交互流程。G-sensor 数据采集方案由用户自行设计与实现。G-sensor 中 ACC 建议配置：≥±4G、采样率(100Hz)。

用户需要添加 G-sensor 采样使能以及获取 G-sensor 数据的代码：

gsensor 操作函数详见附录一。

3.7.2 同步读取

若 G-sensor 的读取时序与 PPG 保持一致，即上述三个 G-sensor 相关的接口函数，在时间轴上的调用位置如下图所示：

汇顶机密
Goodix Confidential For pm@lierda.com Only

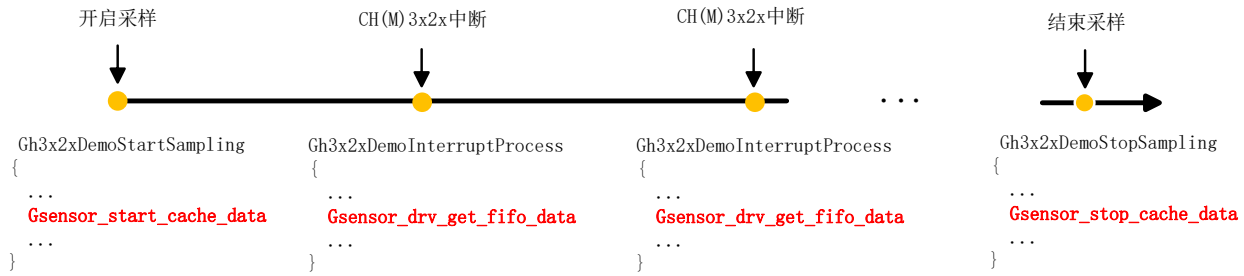


图 3-6 G-sensor 接口同步调用时序

在这种情况下，gh3x2x_demo_config.h 中__GS_NONSYNC_READ_EN__应设置为 0。

3.7.3 异步读取

当用户的工程中，gsesor 的 buffer 读取在其它线程进行，或读取周期与 gh3x2x 线程不同（比 Gsensor 与 PPG 在两个不同的定时器中读取且定时器中断时序不一致），如下图所示：

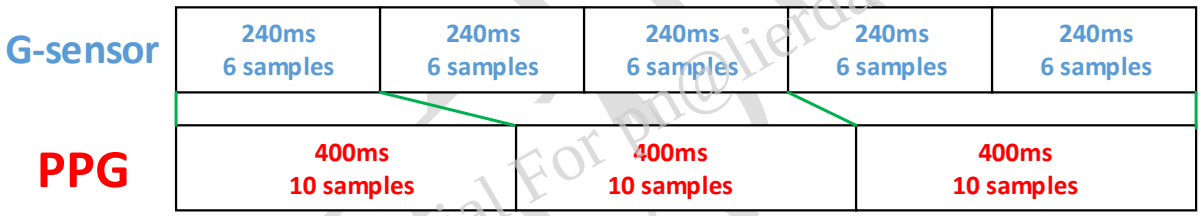


图 3-7 G-sensor 接口异步调用时序

此时认为

异步时序时，每次 PPG 读取中断触发时，读到的 Gsensor 数据会有差异且时序不同步。驱动库中对于这种情况做了时序对齐处理，打开处理需将 gh3x2x_demo_config.h 中__GS_NONSYNC_READ_EN__设置为 1，并设置__GS_SAMPLE_RATE_HZ__、__GS_READ_POINT_NUM__、__GS_READ_POINT_NUM_JITTER__三个值。

3.7.4 时间戳对齐

时间戳对齐方案，参考文档《Goodix 算法调用 SDK 接口说明书》。

3.8 添加配置表

配置表分驱动配置表和算法配置表，如 GTTestTool 工具中窗口显示：

Goodix Confidential For pm@lierda.com only

```
/*-------------- List all registers ---------------------------*/
STGh3x2xReg stGh3x2xRegConfigArr[113] =
{
    {0x0000, 0x0610}, {0x0004, 0x001F}, {0x000A, 0x0064}, {0x000E, 0x000D}, {0x0100, 0x0100}, {0x0102, 0x0302},
    {0x0104, 0x0804}, {0x010A, 0x277C}, {0x0110, 0x0D54}, {0x0112, 0x5555}, {0x0114, 0x00AA}, {0x011E, 0x000C},
    {0x0120, 0x040C}, {0x0122, 0x0CFF}, {0x0126, 0x2744}, {0x0128, 0x0001}, {0x012A, 0x0000}, {0x012C, 0x0C10},
    {0x012E, 0x4442}, {0x0130, 0x0002}, {0x013A, 0x010C}, {0x013C, 0x0400}, {0x0142, 0x2744}, {0x0144, 0x0001},
    {0x0146, 0x0000}, {0x0148, 0x0000}, {0x014A, 0x4442}, {0x014C, 0x0002}, {0x0156, 0x0100}, {0x0158, 0x0400},
    {0x015E, 0x27FC}, {0x0164, 0x0D56}, {0x0168, 0x00AA}, {0x0172, 0x0119}, {0x0174, 0x0400}, {0x0176, 0xFF19},
    {0x017A, 0xC744}, {0x017C, 0x0001}, {0x017E, 0x0000}, {0x0180, 0x0C10}, {0x0182, 0x4442}, {0x0184, 0x0002},
    {0x018A, 0x0003}, {0x018E, 0x010C}, {0x0190, 0x0400}, {0x01EC, 0x0136}, {0x01EE, 0x00AC}, {0x01F0, 0x002E},
    {0x01F2, 0x030E}, {0x01F4, 0x00AC}, {0x0200, 0x0120}, {0x0208, 0x0551}, {0x0280, 0x0A00}, {0x0282, 0xF530},
    {0x0284, 0x9388}, {0x0286, 0xC45C}, {0x0288, 0xA328}, {0x0502, 0x4360}, {0x0504, 0x000A}, {0x0588, 0x0008},
    {0x0692, 0x0233}, {0x0694, 0x0400}, {0x0698, 0x2310}, {0x0700, 0x0000}, {0x1000, 0x0000}, {0x1002, 0x3197},
    {0x1004, 0x0000}, {0x1006, 0xEDAE}, {0x1008, 0x2AE2}, {0x100A, 0x0001}, {0x10E0, 0x0000}, {0x10E2, 0x0000},
    {0x10E4, 0xFF00}, {0x10E6, 0xFFFF}, {0x10E8, 0x0202}, {0x10EA, 0x0202}, {0x10EC, 0x0202}, {0x10EE, 0x0202},
    {0x1120, 0x0064}, {0x1122, 0x0005}, {0x1124, 0x0000}, {0x1140, 0x0001}, {0x1142, 0x00C8}, {0x1144, 0x0001},
    {0x1160, 0x0009}, {0x1162, 0x0000}, {0x1164, 0x0000}, {0x1166, 0x4568}, {0x1168, 0x9940}, {0x116A, 0x00D9},
    {0x116C, 0x8800}, {0x116E, 0x0093}, {0x1170, 0x5C00}, {0x1172, 0x00C4}, {0x1174, 0x2800}, {0x1176, 0x00A3},
    {0x2000, 0x0001}, {0x2002, 0x0082}, {0x2022, 0x0004}, {0x2024, 0x0901}, {0x2026, 0x1911}, {0x2132, 0x0007},
    {0x2134, 0x2201}, {0x2136, 0x6245}, {0x2138, 0x726A}, {0x213A, 0x007A}, {0x2880, 0x0005}, {0x2882, 0x0019},
    {0x2892, 0x0019}, {0x3000, 0x0401}, {0x3002, 0x0000}, {0x3004, 0x49F0}, {0x3006, 0x0082},
    {0xFFFF, 0x0001}
};
```

Driver cfg array

```
STGh3x2xReg stGh3x2xRegConfigArr[10] =
{
    {0x33C0, 0x0000}, {0x33C2, 0x0000}, {0x33C4, 0x0000}, {0x33C6, 0x0000}, {0x33C8, 0x0000}, {0x33CA, 0x0000},
    {0x33CC, 0x0000}, {0x33CE, 0x0000}, {0x33D0, 0x0000}, {0x33D2, 0x0000},{0xFFFF, 0x0001}
};
/*-----------------------------------------*/
```

Algorithm cfg array

3.8.1 驱动配置表的添加

为了简化对 GH(M)3x2x 驱动库寄存器的操作，用户可使用 GH(M)3x2x 配套的 EVK 工具，生成驱动库寄存器配置表，该表中保存了需要对驱动库进行配置的所有寄存器以及要写入的值。该配置表以寄存器数组形式提供给用户，并在例程中自动加载进驱动库。

注：**GTTestTool 2.3.2 版本及以上**的工具生成的 ini 配置文件中，[drvregister-table]字段已包含配置数组，可从文件中直接复制。

汇顶科技 机密信息
未经允许 不得转载

问题反馈

Log Information

```
STGh3x2xReg stGh3x2xRegConfigArr[110] =
{
    {0x0004, 0x001F}, {0x0006, 0x0003}, {0x000A, 0x00C8}, {0x000E, 0x000D}, {0x0100, 0x0100}, {0x0102, 0x0302},
    {0x010A, 0x277C}, {0x0110, 0x01D3}, {0x0112, 0x0000}, {0x011E, 0x004C}, {0x0120, 0x0400}, {0x0122, 0x19FF},
    {0x0126, 0x2744}, {0x0128, 0x0001}, {0x012A, 0x0000}, {0x012C, 0x0193}, {0x012E, 0x4440}, {0x013A, 0x024C},
    {0x013C, 0x0400}, {0x013E, 0x19FF}, {0x0142, 0x2744}, {0x0144, 0x0001}, {0x0146, 0x0000}, {0x0148, 0x0193},
    {0x014A, 0x4440}, {0x0156, 0x014C}, {0x0158, 0x0400}, {0x015A, 0x19FF}, {0x015E, 0x277C}, {0x0164, 0x0014},
    {0x0166, 0x0000}, {0x0172, 0x014C}, {0x0174, 0x0400}, {0x01EC, 0x006B}, {0x01EE, 0x006B}, {0x01F0, 0x006B},
    {0x01F2, 0x00BA}, {0x0200, 0x0120}, {0x0280, 0x0A00}, {0x0282, 0xF530}, {0x0284, 0x9388}, {0x0286, 0xC45C},
    {0x0288, 0xA328}, {0x0408, 0x0001}, {0x0410, 0x39C0}, {0x0412, 0x0083}, {0x0414, 0x39C0}, {0x0416, 0x0083},
    {0x0422, 0x0011}, {0x0424, 0x0901}, {0x0426, 0x0102}, {0x0428, 0x0102}, {0x0502, 0x0000}, {0x0504, 0x000A},
    {0x0588, 0x0008}, {0x0692, 0x0233}, {0x0694, 0x0400}, {0x0698, 0x2310}, {0x0700, 0x0000}, {0x1000, 0x0000},
    {0x1002, 0x3196}, {0x1004, 0x0000}, {0x1006, 0x1B64}, {0x1008, 0x2A32}, {0x100A, 0x0001}, {0x10E0, 0x0000},
    {0x10E2, 0x0000}, {0x10E4, 0x0100}, {0x10E6, 0x0000}, {0x10E8, 0x0202}, {0x10EA, 0x0202}, {0x10EC, 0x0202},
    {0x10EE, 0x0202}, {0x1120, 0x00C8}, {0x1122, 0x0005}, {0x1124, 0x0000}, {0x1140, 0x0000}, {0x1142, 0x00C8},
    {0x1144, 0x0001}, {0x1160, 0x0007}, {0x1162, 0x0000}, {0x1164, 0x0000}, {0x1166, 0x4568}, {0x1168, 0x9940},
    {0x116A, 0x00D9}, {0x116C, 0x8800}, {0x116E, 0x0093}, {0x1170, 0x5C00}, {0x1172, 0x00C4}, {0x1174, 0x2800},
    {0x1176, 0x00A3}, {0x2000, 0x0004}, {0x2002, 0x6A62}, {0x2004, 0x7A72}, {0x2022, 0x0004}, {0x2024, 0x0901},
    {0x2026, 0x1911}, {0x2044, 0x0004}, {0x2046, 0x0901}, {0x2048, 0x1911}, {0x20CC, 0x0002}, {0x20CE, 0x4223},
    {0x2880, 0x0005}, {0x2882, 0x0019}, {0x2884, 0x0019}, {0x288C, 0x0019}, {0x3000, 0x0001}, {0x3002, 0x0000},
    {0x3004, 0x39C0}, {0x3006, 0x0083},{0xFFFF, 0x0001}
};
```

Save OK

图 3-8 工具 decode 出来的驱动库配置表格式

HR+HRV+SPO2+ADTmixtest.ini

```
[drvregister-table]
000="STGh31xxReg stGh31xxRegConfigArr[110] ="
001={
002="    {0x0004, 0x001F}, {0x0006, 0x0003}, {0x000A, 0x00C8}, {0x000E, 0x000D}, {0x0100, 0x0100}, {0x0102, 0x0302},"
003="    {0x010A, 0x277C}, {0x0110, 0x01D3}, {0x0112, 0x0000}, {0x011E, 0x004C}, {0x0120, 0x0400}, {0x0122, 0x19FF},"
004="    {0x0126, 0x2744}, {0x0128, 0x0001}, {0x012A, 0x0000}, {0x012C, 0x0193}, {0x012E, 0x4440}, {0x013A, 0x024C},"
005="    {0x013C, 0x0400}, {0x013E, 0x19FF}, {0x0142, 0x2744}, {0x0144, 0x0001}, {0x0146, 0x0000}, {0x0148, 0x0193},"
006="    {0x014A, 0x4440}, {0x0156, 0x014C}, {0x0158, 0x0400}, {0x015A, 0x19FF}, {0x015E, 0x277C}, {0x0164, 0x0014},"
007="    {0x0166, 0x0000}, {0x0172, 0x014C}, {0x0174, 0x0400}, {0x01EC, 0x006B}, {0x01EE, 0x006B}, {0x01F0, 0x006B},"
008="    {0x01F2, 0x00BA}, {0x0200, 0x0120}, {0x0280, 0x0A00}, {0x0282, 0xF530}, {0x0284, 0x9388}, {0x0286, 0xC45C},"
009="    {0x0288, 0xA328}, {0x0408, 0x0001}, {0x0410, 0x39C0}, {0x0412, 0x0083}, {0x0414, 0x39C0}, {0x0416, 0x0083},"
010="    {0x0422, 0x0011}, {0x0424, 0x0901}, {0x0426, 0x0102}, {0x0428, 0x0102}, {0x0502, 0x4F66}, {0x0504, 0x000A},"
011="    {0x0588, 0x0008}, {0x0692, 0x0233}, {0x0694, 0x0400}, {0x0698, 0x2310}, {0x0700, 0x0000}, {0x1000, 0x0000},"
012="    {0x1002, 0x3196}, {0x1004, 0x0000}, {0x1006, 0xF210}, {0x1008, 0x2A31}, {0x100A, 0x0001}, {0x10E0, 0x0000},"
013="    {0x10E2, 0x0000}, {0x10E4, 0x0100}, {0x10E6, 0x0000}, {0x10E8, 0x0202}, {0x10EA, 0x0202}, {0x10EC, 0x0202},"
014="    {0x10EE, 0x0202}, {0x1120, 0x00C8}, {0x1122, 0x0005}, {0x1124, 0x0000}, {0x1140, 0x0000}, {0x1142, 0x00C8},"
015="    {0x1144, 0x0001}, {0x1160, 0x0007}, {0x1162, 0x0000}, {0x1164, 0x0000}, {0x1166, 0x4568}, {0x1168, 0x9940},"
016="    {0x116A, 0x00D9}, {0x116C, 0x8800}, {0x116E, 0x0093}, {0x1170, 0x5C00}, {0x1172, 0x00C4}, {0x1174, 0x2800},"
017="    {0x1176, 0x00A3}, {0x2000, 0x0004}, {0x2002, 0x6A62}, {0x2004, 0x7A72}, {0x2022, 0x0004}, {0x2024, 0x0901},"
018="    {0x2026, 0x1911}, {0x2044, 0x0004}, {0x2046, 0x0901}, {0x2048, 0x1911}, {0x20CC, 0x0002}, {0x20CE, 0x4223},"
019="    {0x2880, 0x0005}, {0x2882, 0x0019}, {0x2884, 0x0019}, {0x288C, 0x0019}, {0x3000, 0x0001}, {0x3002, 0x0000},"
020="    {0x3004, 0x39C0}, {0x3006, 0x0083},{0xFFFF, 0x0001}"
021="};"
```

图 3-9 配置 ini 文件中的驱动库配置表格式

用户可将工具生成的寄存器数组内容或工具生成的 ini 配置文件中的[drvregister-table]字段内容，拷贝到“gh3x2x_demo_reg_array.c”文件中。

```
const STGh3x2xReg gh3x2x_reg_list0[] =
{
    {0x0004, 0x001F}, {0x0006, 0x0003}, {0x000A, 0x00C8}, {0x000E, 0x000D}, {0x0100, 0x0100}, {0x0102, 0x0302},
    {0x010A, 0x277C}, {0x0110, 0x01D3}, {0x0112, 0x0000}, {0x011E, 0x004C}, {0x0120, 0x0400}, {0x0122, 0x19FF},
    {0x0126, 0x2744}, {0x0128, 0x0001}, {0x012A, 0x0000}, {0x012C, 0x0193}, {0x012E, 0x4440}, {0x013A, 0x024C},
    {0x013C, 0x0400}, {0x013E, 0x19FF}, {0x0142, 0x2744}, {0x0144, 0x0001}, {0x0146, 0x0000}, {0x0148, 0x0193},
    {0x014A, 0x4440}, {0x0156, 0x014C}, {0x0158, 0x0400}, {0x015A, 0x19FF}, {0x015E, 0x277C}, {0x0164, 0x0014},
    {0x0166, 0x0000}, {0x0172, 0x014C}, {0x0174, 0x0400}, {0x01EC, 0x006B}, {0x01EE, 0x006B}, {0x01F0, 0x006B},
    {0x01F2, 0x00BA}, {0x0200, 0x0120}, {0x0280, 0x0A00}, {0x0282, 0xF530}, {0x0284, 0x9388}, {0x0286, 0xC45C},
    {0x0288, 0xA328}, {0x0408, 0x0001}, {0x0410, 0x39C0}, {0x0412, 0x0083}, {0x0414, 0x39C0}, {0x0416, 0x0083},
    {0x0422, 0x0011}, {0x0424, 0x0901}, {0x0426, 0x0102}, {0x0428, 0x0102}, {0x0502, 0x0000}, {0x0504, 0x000A},
    {0x0588, 0x0008}, {0x0692, 0x0233}, {0x0694, 0x0400}, {0x0698, 0x2310}, {0x0700, 0x0000}, {0x1000, 0x0000},
    {0x1002, 0x3196}, {0x1004, 0x0000}, {0x1006, 0x1B64}, {0x1008, 0x2A32}, {0x100A, 0x0001}, {0x10E0, 0x0000},
    {0x10E2, 0x0000}, {0x10E4, 0x0100}, {0x10E6, 0x0000}, {0x10E8, 0x0202}, {0x10EA, 0x0202}, {0x10EC, 0x0202},
    {0x10EE, 0x0202}, {0x1120, 0x00C8}, {0x1122, 0x0005}, {0x1124, 0x0000}, {0x1140, 0x0000}, {0x1142, 0x00C8},
    {0x1144, 0x0001}, {0x1160, 0x0007}, {0x1162, 0x0000}, {0x1164, 0x0000}, {0x1166, 0x4568}, {0x1168, 0x9940},
    {0x116A, 0x00D9}, {0x116C, 0x8800}, {0x116E, 0x0093}, {0x1170, 0x5C00}, {0x1172, 0x00C4}, {0x1174, 0x2800},
    {0x1176, 0x00A3}, {0x2000, 0x0004}, {0x2002, 0x6A62}, {0x2004, 0x7A72}, {0x2022, 0x0004}, {0x2024, 0x0901},
    {0x2026, 0x1911}, {0x2044, 0x0004}, {0x2046, 0x0901}, {0x2048, 0x1911}, {0x20CC, 0x0002}, {0x20CE, 0x4223},
    {0x2880, 0x0005}, {0x2882, 0x0019}, {0x2884, 0x0019}, {0x288C, 0x0019}, {0x3000, 0x0001}, {0x3002, 0x0000},
    {0x3004, 0x39C0}, {0x3006, 0x0083}, {0xFFFF, 0x0001}
};
```

图 3-10 添加配置文件到例程中

上图是例程中自带的一个配置表，用户使用新的配置表替换掉即可。

目前例程支持最多加入 8 个配置表，用户可配置“__GH3X2X_CFG_LIST_MAX_NUM__”来增加或者减少配置表的个数。如果用户需要更多配置表（大于 8 个），可修改“__GH3X2X_CFG_LIST_MAX_NUM__”，并参照“gh3x2x_demo_reg_array.c”文件中已有的配置表，手动添加新的配置表，并将该配置表添加到数组“g_stGh3x2xCfgListArr []”中即可。

建议用户添加的所有配置表，每个配置表中，当开启的功能组合仅仅存在于其中一个 cfg 数组中，比如需要开启的功能组合是 SPO2+ADT，配置表 0 包含 HR+ADT、配置表 1 包含 SPO2+ADT、配置表 3 包含 ECG+ADT，此时可将“__GH3X2X_ARRAY_CFG_MANUAL_SWITCH_EN__”配置为 0，在开启采样时，例程中会根据采样功能，自动切换配置表，用户无需手动切换。

以上即完成了用户需要添加的代码部分。

3.8.2 算法配置表的添加

算法配置表的添加方式和驱动配置表类似，添加的位置是在 gh3x2x_demo_algo_reg_array.c。

注意事项：驱动配置表需要和算法配置表一一对应！即同一个 ini 文件中的驱动配置表和算法配置表需要放在同一个索引位置。如某个 ini 文件的驱动配置放在 **gh3x2x_reg_list2** 的位置，那么算法配置表一定要放在 **gh3x2x_algo_reg_list2** 的位置；

3.9 驱动和算法分开运行的修改

如果用户想让驱动和算法运行在两个线程或是两个 CPU 内核中，那么需要做额外的修改。具体修改详见“驱动和算法分开运行详细修改指引”章节。

如果用户采用常规方式运行驱动和算法，请跳过本接修改；

3.10 初始化接口调用

初始化时，调用“Gh3x2xDemoInit()”，若返回值为 SUCCESS，即可完成对驱动库的初始化。该接口内部会完成对 SPI/I2C 接口、驱动库、IO 管脚等功能的初始化。由于例程代码不包含 G-sensor 的初始化，因此，用户需要单独初始化 G-sensor 模块。

初始化 SPI 时序示例如下：

GH(M)3X2X初始化时序.csv

3.11 采样控制

3.11.1 开始和停止

开启 GH(M)3x2x 采样时，调用“Gh3x2xDemoStartSampling()”。函数形参代表需要开启的功能，目前支持 ADT、HR、SPO2、ECG 功能的采样，可通过相或的方式，一次性开启多个功能。

停止 GH(M)3x2x 采样时，调用“Gh3x2xDemoStopSampling()”。函数形参代表需要关闭的功能，可通过相或的方式，一次性关闭多个功能。

3.11.2 采样率的预设定

各功能的默认采样率由驱动库配置表决定，另外，也可以在开启采样之前，调用 Gh3x2xDemoFunctionSampleRateSet，来设定对应功能的采样率。

3.11.3 采样通道使能的预设定

各功能的默认使能的通道由驱动库配置表决定，另外，也可以在开启采样之前，调用 Gh3x2xDemoFunctionChannelEnSet，来设定对应功能的使能通道。未使能的通道将不开启采样，以节省功耗；

汇顶机密
Goodix Confidential For pn@lierda.com Only

汇顶科技 机密信息
未经允许 不得转载
问题反馈

3.11.4 Fifo 中断水线设定

在中断响应模式下，Fifo 在满足水线的情况下才会上报中断，设置 Fifo 水线的函数说明详见附录一中的 GH3X2X_FifoWatermarkThrConfig 调用说明。

注意，当 Fifo 内数据量超过水线而应用未响应中断时（即未读走数据），Fifo 水线中断不会上报，转而准备上报 Fifo Full 中断。

水线设定分为静态设定和动态设定。静态设定是在所有功能全部 stop 后，调用 GH3X2X_FifoWatermarkThrConfig 配置水线；

动态设定需要遵守的操作步骤如下：

1、调用 GH3X2X_FifoWatermarkThrConfig 改变水线（注意和其他 GH3X2X 接口保持互斥运行）；
2、调用 GH3X2X_SetSoftEvent(GH3X2X_SOFT_EVENT_NEED_FORCE_READ_FIFO)告诉驱动层准备强制读取一次 fifo（避免切换水线后，芯片不上报水线中断）
3、触发一次 Gh3x2xDemoInterruptProcess 的执行；

3.12 GH3x2x 中断处理

用户如果使用轮询方式和 GH(M)3x2x 进行交互，需要设置“__INTERRUPT_PROCESS_BY_POLLING__”为 1，同时，在轮询时间到来时，调用“Gh3x2xDemoInterruptProcess”进行处理。

用户如需处理中断事件，需要在产生硬件中断时，调用“hal_gh3x2x_int_handler_call_back”。

如果用户使用了操作系统，可在“hal_gh3x2x_int_handler_call_back”中添加发送中断事件的代码，在处理 GH(M)3x2x 中断事件的线程中，调用“Gh3x2xDemoInterruptProcess”进行中断事件的处理。

如果用户没有使用操作系统，在主循环中，调用“Gh3x2xDemoInterruptProcess”进行中断事件的处理。

汇顶机密 GoodixConfidential For pn@lierda.com only

3.12.1 中断模式动态切换

需要将宏__INTERRUPT_PROCESS_BY_POLLING__切换为(__POLLING_INT_PROCESS_MODE__)，操作流程如下：

1) 调用 Gh3x2xDemoStopSampling 函数停止当前操作功能。
2) 调用 Gh3x2xInterruptModeSwitch 函数切换中断模式
3) 调用 Gh3x2xDemoInit 重新初始化，此时可开启轮询定时器或是初始化中断引脚
4) 调用 Gh3x2xDemoStartSampling 开启对应功能，或进行其他操作

Gh3x2xInterruptModeSwitch 函数与 Gh3x2xGetInterruptMode 函数说明见附录一。

3.13 外层接口调用流程的设计和编写

3.13.1 带操作系统的轮询模式

带操作系统的轮询模式需要配置 __PLATFORM_WITHOUT_OS__ = 0，__INTERRUPT_PROCESS_BY_POLLING__ = 1 ；

主流程执行流程可参考如下：

汇顶机密
Goodix Confidential For pn@lieda.com Only

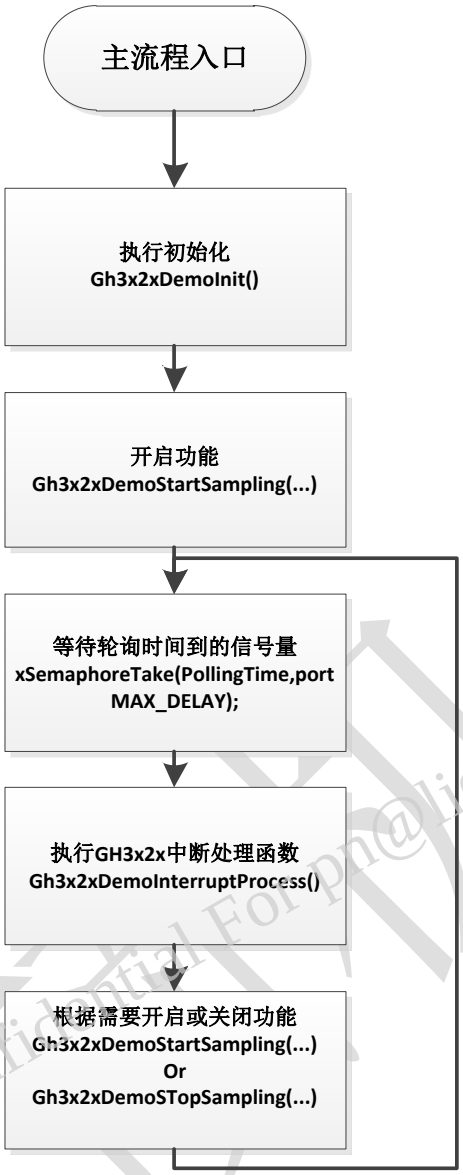


图 3-11 主流程执行流程

3.13.2 不带操作系统的轮询模式

带操作系统的轮询模式需要配置 __PLATFORM_WITHOUT_OS__ = 1，__INTERRUPT_PROCESS_BY_POLLING__ = 1 ；

主流程执行流程可参考如下：

Goodix Confidential For pn@lierda.com Only

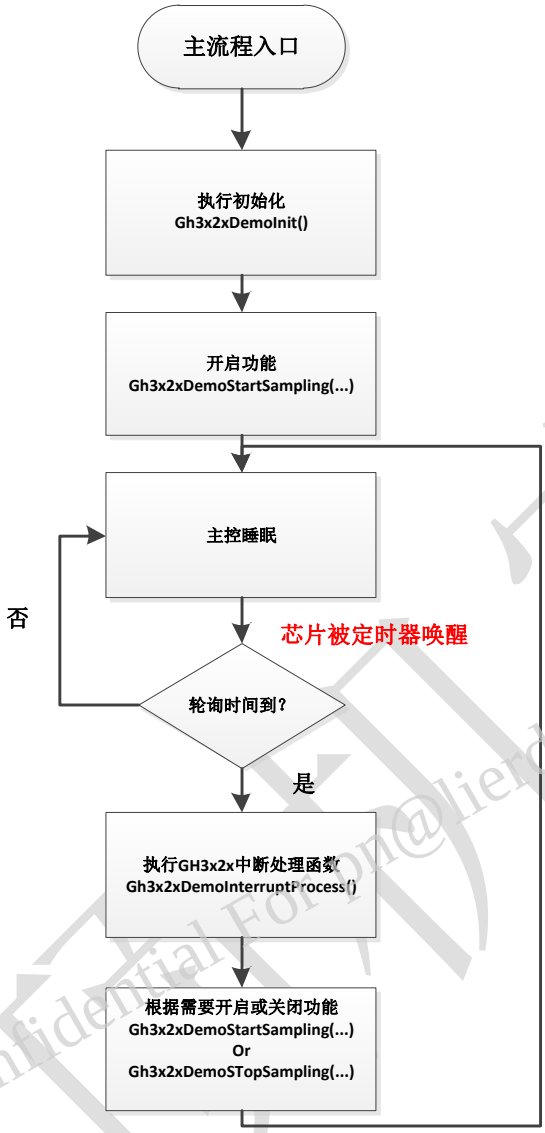


图 3-12 主流程执行流程

3.13.3 带操作系统的中断模式

带操作系统的轮询模式需要配置 __PLATFORM_WITHOUT_OS__ = 0，__INTERRUPT_PROCESS_BY_POLLING__ = 0 ；

主流程执行流程可参考如下：

Goodix Confidential For on@lierda.com Only

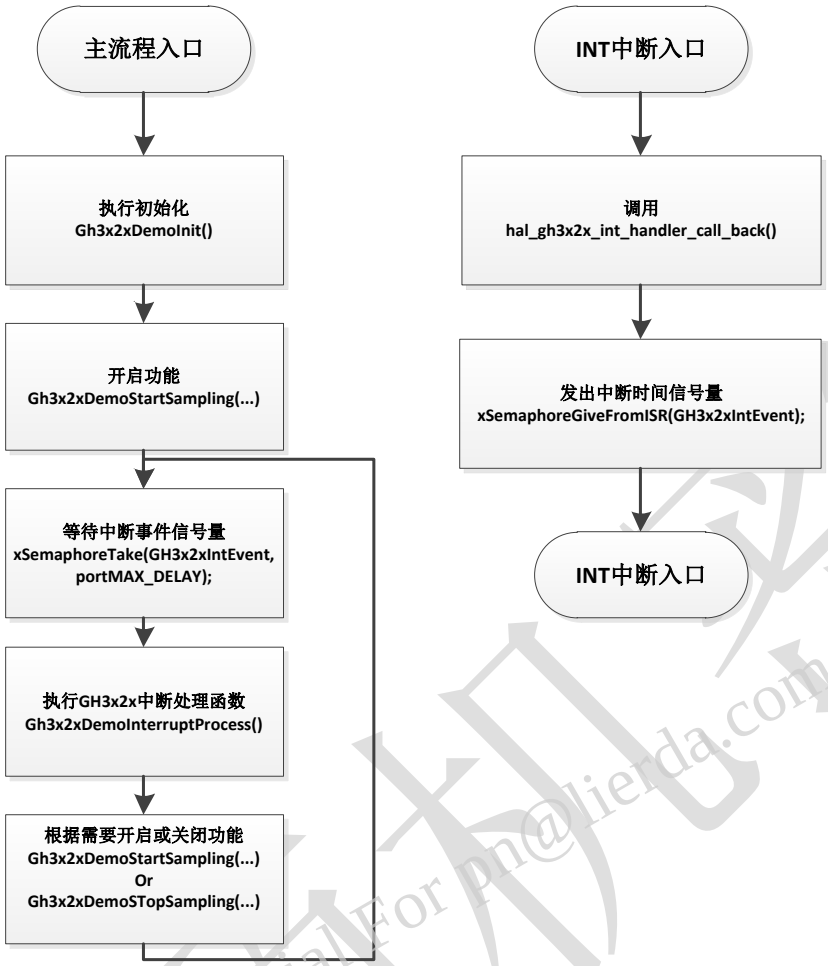


图 3-13 主流程执行流程

3.13.4 不带操作系统的中断模式

带操作系统的轮询模式需要配置 __PLATFORM_WITHOUT_OS__ = 1，__INTERRUPT_PROCESS_BY_POLLING__ = 0 ；

主流程执行流程可参考如下：

汇顶机密 Goodix Confidential For pn@lierda.com Only

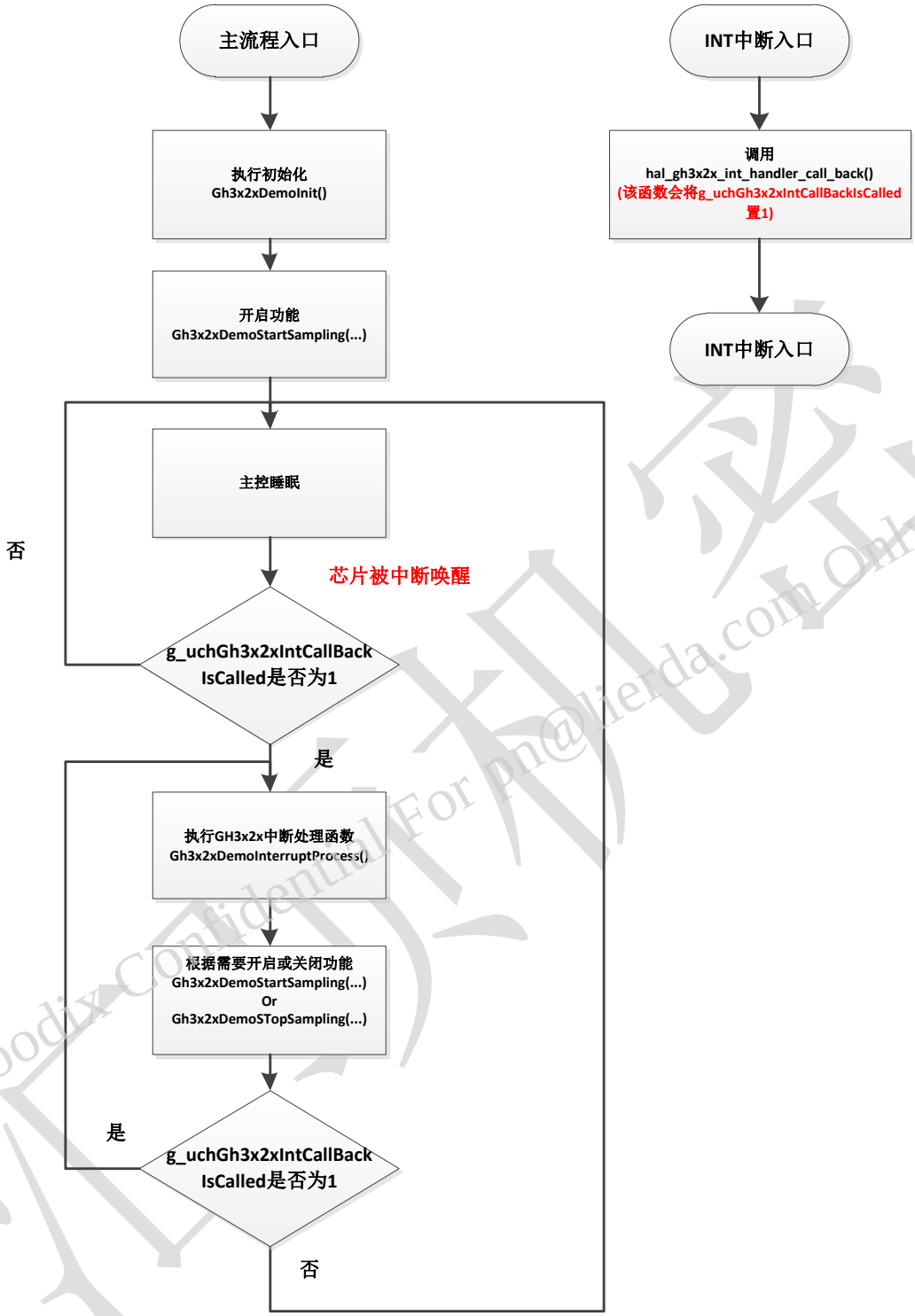


图 3-14 主流程执行流程

3.14 佩戴检测定时器处理

佩戴检测定时器中断产生后需要调用其处理函数：“Gh3x2xDemoMoveDetectTimerHandler”；具体流程参考**第 4 章**。

3.15 应用模式切换

将驱动库配置表中包含的所有采样功能看成是驱动库的一种应用模式，比如某个配置表只包含了 HR 功能，某个配置表中包含了 HR 和 SPO2 功能，这两个配置表可以看成是两种不同的应用模式。

每种应用模式都对应着一个驱动库配置表，该配置表由用户保存在“gh3x2x_demo_reg_array.c”文件中。

在初始化驱动库时，程序会默认将“gh3x2x_reg_list0”配置表加载进驱动库中。

如果用户将“__GH3X2X_ARRAY_CFG_MANUAL_SWITCH_EN__”配置为 0，用户无需手动切换应用模式，例程中会自动切换配置表。PS：如果用户想要开启的功能组合存在于多张配置表，搜寻顺序是：当前加载的配置表->配置表 0->配置表 1…->配置 7；直到搜索到符合要求的配置表。因此在这种情况下，建议将__GH3X2X_ARRAY_CFG_MANUAL_SWITCH_EN__设置为 1；

如果用户将“__GH3X2X_ARRAY_CFG_MANUAL_SWITCH_EN__”配置为 1，用户在开启功能前，通过调用函数“Gh3x2xDemoArrayCfgSwitch”进行切换。该函数入参为配置表索引号，驱动库会停掉当前正在运行的功能并复位 GH(M)3x2x 芯片，之后会将用户指定的配置表加载到驱动库中。即使手动切换的配置表中不包含用户想要开启的功能，驱动库也不会自动搜寻或加载其他配置表，而是在驱动库 debug 版本中上报错误 log；

3.16 驱动 Hook 功能

为了方便用户在驱动库处理流程的关键节点做拓展处理，驱动库提供了若干 hook 功能（用户可理解为回调函数），按功能划分可分为两类：

采样流程类 hook 函数见附录一

用户如需使用此类函数，需要将“__SUPPORT_HOOK_FUNC_CONFIG__”设置为 1，并在“gh3x2x_demo_hook.c”文件中完成相关 hook 函数的代码实现。

Goodix 跑库格式范例如下附件：

goodix跑库格式范例.csv

3.17 工程模式

工程模式下，用户可以调用特殊的启动采样的接口，基于 gh3x2x_demo_reg_array.c 中的配置表，修改采样参数（包括采样率、积分时间、TIA gain、LED 电流），并启动采样。用户需要将“__SUPPORT_ENGINEERING_MODE__”设置为 1，驱动库才支持工程模式；

工程模式启动/结束函数详见附录一。

工程模式输入结构体相关参数说明详见附录三。

3.17.1 应用示例

用户需要在工程模式下启动 HR+ADT 功能，并修改 HR、ADT 功能的采样参数。用户需要做的操作如下：

1、 定义采样参数结构体数组：

```
STGh3x2xEngineeringModeSampleParam g_stTestEngineeringModeSamplParmGroup[2];
```

2、 设定采样参数（对采样参数结构体赋值）：

```
g_stTestEngineeringModeSamplParmGroup[0].uchFuncitonId = GH3X2X_FUNCTION_HR;    //set 39unction
g_stTestEngineeringModeSamplParmGroup[0].uchIntTimeChangeEn = 1;    //if you need change int time, set 1
g_stTestEngineeringModeSamplParmGroup[0].uchIntTime = GH3X2X_ENGINEERING_MODE_INT_TIME_316_US;
g_stTestEngineeringModeSamplParmGroup[0].uchSampleRateChangeEn = 1;
g_stTestEngineeringModeSamplParmGroup[0].usSampleRate = 50;
g_stTestEngineeringModeSamplParmGroup[0].uchTiaGainChangeEn = 1;
g_stTestEngineeringModeSamplParmGroup[0].uchTiaGain[0] = GH3X2X_ENGINEERING_MODE_TIA_GAIN_50_K;
g_stTestEngineeringModeSamplParmGroup[0].uchTiaGain[1] =
GH3X2X_ENGINEERING_MODE_TIA_GAIN_1250_K;
g_stTestEngineeringModeSamplParmGroup[0].uchLedCurrentChangeEn = 1;
g_stTestEngineeringModeSamplParmGroup[0].uchLedDrv0Current[0] = 20;
g_stTestEngineeringModeSamplParmGroup[0].uchLedDrv0Current[1] = 20;
g_stTestEngineeringModeSamplParmGroup[0].uchLedDrv1Current[0] = 100;
g_stTestEngineeringModeSamplParmGroup[0].uchLedDrv1Current[1] = 100;


g_stTestEngineeringModeSamplParmGroup[1].uchFuncitonId = GH3X2X_FUNCTION_ADT;
g_stTestEngineeringModeSamplParmGroup[1].uchIntTimeChangeEn = 1;
g_stTestEngineeringModeSamplParmGroup[1].uchIntTime = GH3X2X_ENGINEERING_MODE_INT_TIME_79_US;
g_stTestEngineeringModeSamplParmGroup[1].uchSampleRateChangeEn = 1;
g_stTestEngineeringModeSamplParmGroup[1].usSampleRate = 10;
g_stTestEngineeringModeSamplParmGroup[1].uchTiaGainChangeEn = 1;
g_stTestEngineeringModeSamplParmGroup[1].uchTiaGain[0] = GH3X2X_ENGINEERING_MODE_TIA_GAIN_250_K;
g_stTestEngineeringModeSamplParmGroup[1].uchTiaGain[1] =
GH3X2X_ENGINEERING_MODE_TIA_GAIN_2000_K;
```

汇顶科技 机密信息
未经允许 不得转载

问题反馈

```
g_stTestEngineeringModeSamplParmGroup[1].uchLedCurrentChangeEn = 1;
g_stTestEngineeringModeSamplParmGroup[1].uchLedDrv0Current[0] = 200;
g_stTestEngineeringModeSamplParmGroup[1].uchLedDrv0Current[1] = 200;
g_stTestEngineeringModeSamplParmGroup[1].uchLedDrv1Current[0] = 5;
g_stTestEngineeringModeSamplParmGroup[1].uchLedDrv1Current[1] = 5;
```

3、调用工程模式下的开启采样

Gh3x2xDemoStartSamplingForEngineeringMode(GH3X2X_FUNCTION_HR|GH3X2X_FUNCTION_ADT,g_stTestEngineeringModeSamplParmGroup,2);

4、 采集数据后，在需要停止采样的时候，调用 Gh3x2xDemoStopSamplingForEngineeringMode

3.17.2 注意

1、工程模式下的采样一旦开启，驱动库自动调光、调 Gain 功能会自动被禁用；
2、LED 电流的设定按照数据通道的概念来设定的。如果存在两个通道共用一个 slot，那么最终设定的电流以索引值大的通道为准；

3.18 Demo code 文件修改

除了“gh3x2x_demo_config.h”、“gh3x2x_demo_hook.c”、“gh3x2x_demo_reg_array.c”、“gh3x2x_demo_user.c”，**其他文件中的内容不要改动。**

3.19 线程布置

由于“Gh3x2xDemoStartSampling”、“Gh3x2xDemoStopSampling”、“Gh3x2xDemoProtocolProcess”、“Gh3x2xDemoInit”、“Gh3x2xDemoInterruptProcess”以及“Gh3x2xDemoMoveDetectTimerHandler”均会用到 GH3x2x 驱动库内部函数，并涉及到 Gh3x2x 芯片操作。因此以上函数**必须放入一个线程，如果要放入多个线程，必须对他们做加锁互斥**，防止资源冲突；

3.20 其他

不建议用户对 GH(M)3x2x 进行复位操作。如果用户对 GH(M)3x2x 进行了复位操作，需要调用“Gh3x2xDemoModeSwitch”将配置表加载进驱动库。

汇顶科技 机密信息
未经允许 不得转载

问题反馈

4 佩戴检测模块移植

佩戴检测模块分硬件检测和软件检测两种。硬件检测仅仅能检测到佩戴和对空脱落两种事件。软件检测可以检测出佩戴、由于对空的脱落、由于非活体的脱落三种事件

4.1 硬件佩戴检测移植步骤

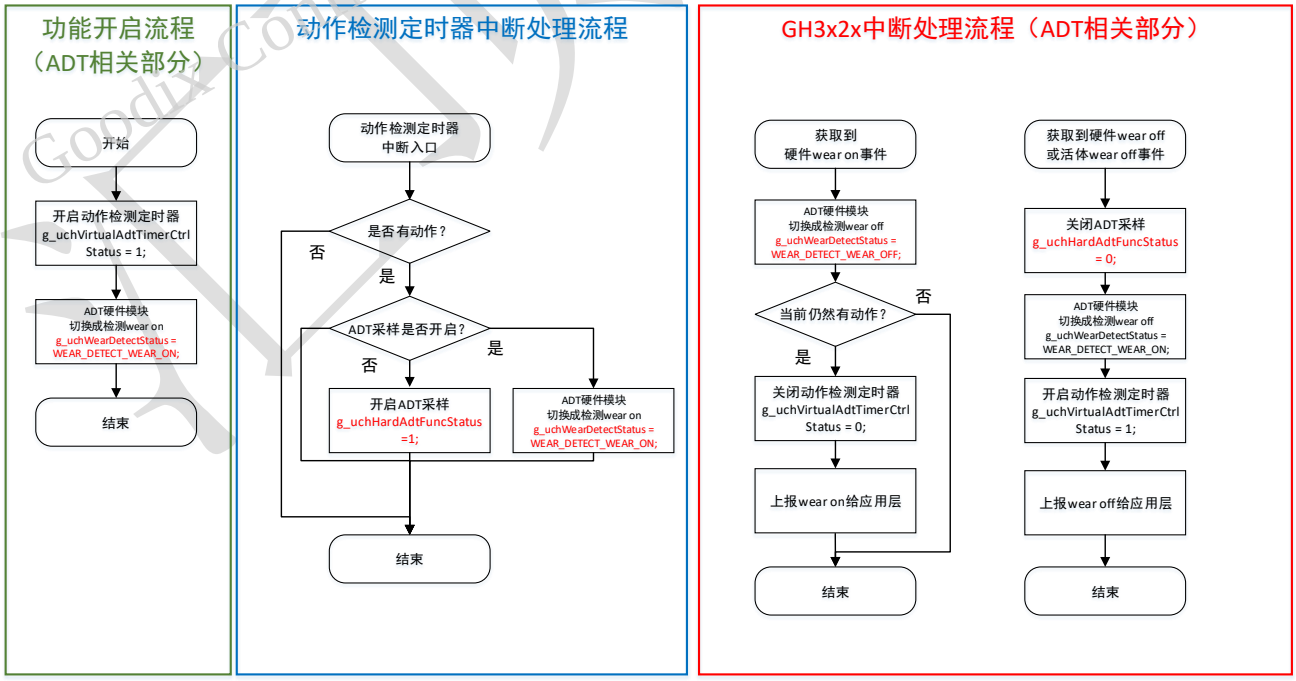
4.1.1 修改宏配置

1）将__SUPPORT_HARD_ADT_CONFIG__ 置为 1

2）将__FUNC_TYPE_SOFT_ADT_ENABLE__ 置为 0

4.2 软件佩戴检测流程图说明

下图为软件佩戴检测流程图，该流程已经在 gh3x2x_demo_soft_adt.c 中实现。该功能需要用到 gsensor 来检测动作。



汇顶机密 Goodix Confidential For pn@lierda.com Only

4.3 软件佩戴检测移植步骤

4.3.1 修改宏配置

1）将__SUPPORT_HARD_ADT_CONFIG__ 置为 1
2）将__FUNC_TYPE_SOFT_ADT_ENABLE__ 置为 1，并确保心率功能是开启状态（即__FUNC_TYPE_HR_ENABLE__为 1）
3）根据实际情况调整以下宏定义

宏	可配置值	描述
__GSENSOR_MOVE_THRESHOLD__	建议值 65	可判定为运动的阈值（sqrt(x)）
__GSENSOR_MOVE_CNT_THRESHOLD__	建议值为 1	连续超过多少次运动阈值可视为检测到了运动
__GSENSOR_NOT_MOVE_CNT_THRESHOLD__	建议值为 ACC 采样率的 0.8	连续小于多少次运动阈值可视为检测到了静止
__USE_POLLING_TIMER_AS_ADT_TIMER__	1（推荐）	用轮询定时器作为运动检测定时器
	0	不用轮询定制器作为运动检测定时器（用户需要额外提供定时器）
__SOFT_ADT_CTRL_WEAR_OFF_ENABLE__	0	当算法或 ACC 检测输出非活体时，是否自动插入 wearoff 中断
	1（推荐）	

4.3.2 添加用户代码以及增加接口调用

1）实现 Gh3x2x_CreateAdtConfirmTimer 函数（在 gh3x2x_demo_user.c 中），用于创建佩戴检测确认的定时器（定时周期建议值为 50ms）；
2）实现 Gh3x2x_StartAdtConfirmTimer 函数（在 gh3x2x_demo_user.c 中），用于开启佩戴检测确认的定时器；

汇顶科技 机密信息
未经允许 不得转载

问题反馈

3）实现 Gh3x2x_StopAdtConfirmTimer 函数（在 gh3x2x_demo_user.c 中），用于开启佩戴检测确认的定时器；

4）在佩戴检测定时器中断中调用 Gh3x2xDemoMoveDetectTimerHandler

注意：如果驱动库被设定为轮询模式（__INTERRUPT_PROCESS_BY_POLLING__ = 1），且 __USE_POLLING_TIMER_AS_ADT_TIMER__ 配置为 1，则无需配置上述定时器相关函数，也无需调用 Gh3x2xDemoMoveDetectTimerHandler

4.4 活体检测函数应用示例

以下为推荐的活体检测方案的应用示例（请保证配置中包含 SOFT_ADT_GREEN/SOFT_ADT_IR/ADT/若干主功能）：

1、开启 ADT 功能，ADT 功能包括 Gsensor 检测与硬件 ADT 结合的动作检测方案流程，流程为：

1）检测到 Gsensor 动作，开启硬件 ADT 采样

2）检测 Gsensor 动作与 ADT 检测同时进行，都符合 wear on 条件时，中断函数接受到 wear on 中断

2、检测到 wear on，用户可在 **Gh3x2x_WearEventHook** 函数中获得该中断，并在 hook 函数中开启红外 SOFT_ADT_IR 功能

3、通过算法调用模块，SOFT_ADT_IR 算法的 hook 函数 **GH3X2X_SoftAdtIrAlgorithmResultReport**（参考文档《Goodix 算法调用 SDK 接口说明书》）获取 SOFT_ADT_IR 算法结果

注：该函数中，若收到的结果为脱落/非活体，可调用 GH3X2X_SetSoftEvent(GH3X2X_SOFT_EVENT_WEAR_OFF)；之后，下一次中断函数执行时会由软件写入 wear off 硬件中断，客户会在 Gh3x2x_WearEventHook 函数中获得 wear off 中断，执行相关操作。

4、获取到算法返回活体佩戴标志，关闭 SOFT_ADT_IR 功能，开启主功能 + 绿光 SOFT_ADT_GREEN，例如：

Gh3x2xDemoStopSampling(GH3X2X_FUNCTION_SOFT_ADT_IR);

Gh3x2xDemoStartSampling(GH3X2X_FUNCTION_SOFT_ADT_GREEN|GH3X2X_FUNCTION_HR);

5、通过算法调用模块，SOFT_ADT_GREEN 算法的 hook 函数 **GH3X2X_SoftAdtGreenAlgorithmResultReport**（参考文档《Goodix 算法调用 SDK 接口说明书》）获取 SOFT_ADT_GREEN 算法结果

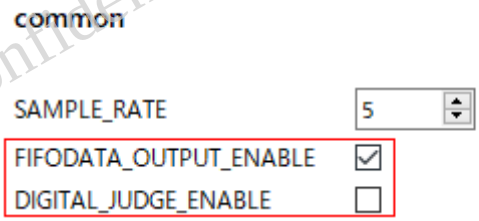
注：该函数中，若收到的结果为脱落/非活体，可调用 GH3X2X_SetSoftEvent(GH3X2X_SOFT_EVENT_WEAR_OFF)；之后，下一次中断函数执行时会由软件写入 wear off 硬件中断，用户会在 Gh3x2x_WearEventHook 函数中获得 wear off 中断，执行相关操作。

4.5 用软件来代替硬件检测模块

部分形态的产品（如指夹仪、指环等）无法使用芯片内部的硬件模块判断佩戴，因此需要将 ADT 的数据（一般是红外 5Hz）输出并通过软件来判断佩戴。

4.5.1 用软件来代替硬件检测模块的移植步骤

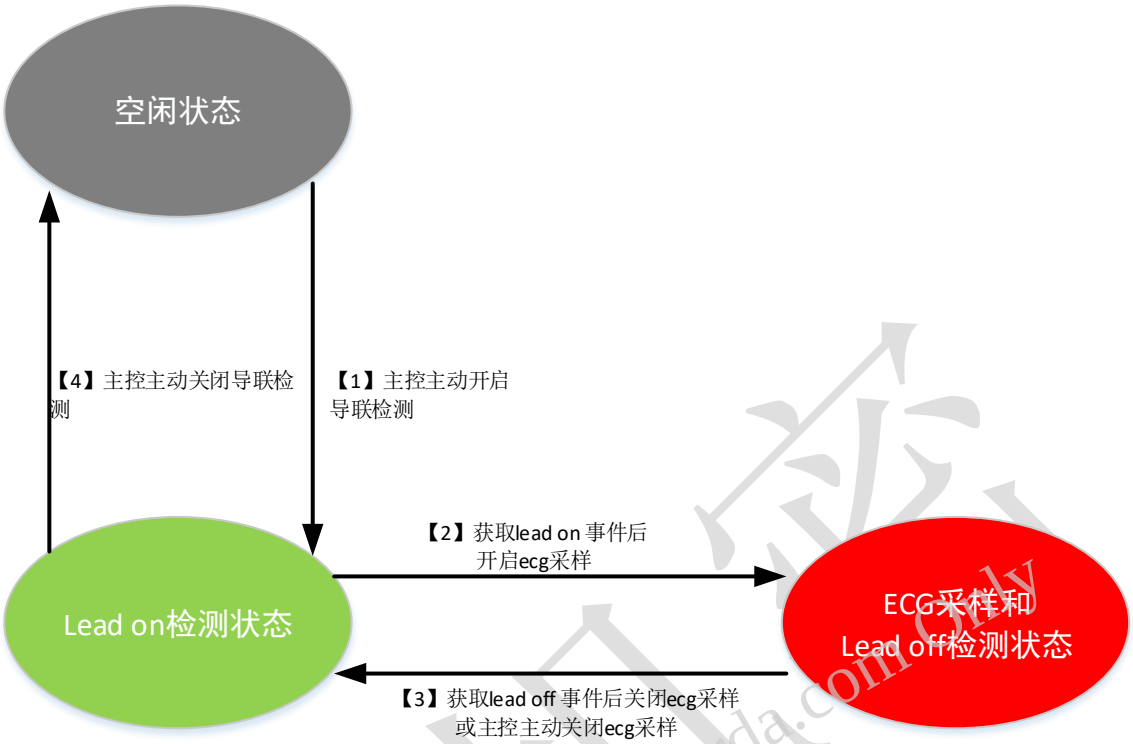
1）将__HARD_ADT_ALGO_CHECK_EN__ 设置为 1
2）如果 mcu 采用中断方式处理 sensor 数据，设置_ADT_ONLY_PARTICULAR_WM_CONFIG__为 5（5 为典型值，考虑到 ADT 一般为 5Hz，判断周期为 1s）；
3）如果 mcu 采用轮询方式处理 sensor 数据，设置_ADT_ONLY_PARTICULAR_WM_CONFIG__为 0；
4）根据光路特性设置__HARD_ADT_WAER_ON_THRD__ 和__HARD_ADT_WAER_OFF_THRD__（在 gh3x2x_demo_algo_call_adt.c 中）；一般的，反射式光路（如手环类），__HARD_ADT_WAER_ON_THRD__的值大于__HARD_ADT_WAER_OFF_THRD__；投射式光路（如指夹仪）__HARD_ADT_WAER_ON_THRD__的值小__HARD_ADT_WAER_OFF_THRD__；
5）修改 gh3x2x_demo_reg_array.c 的配置表，配置工具的配置项修改如下：
   a) 勾选 ADT->common-> FIFODATA_OUTPUT_ENABLE
   b) 不勾选 ADT->common->DIGITAL_JUDGE_ENABLE



5 ECG 功能使用说明

5.1 推荐的控制流程

控制流程如下图：



图中的各个步骤的具体操作方法描述如下：

【1】 **主控主动开启导联检测：**通过调 Gh3x2xDemoStartSampling(GH3X2X_FUNCTION_LEAD_DET)开启导联检测；

【2】 **获取 lead on 事件后开启 ecg 采样；**获取 lead on 事件是通过 Gh3x2x_LeadOnEventHook 函数获取；ecg 采样时通过 Gh3x2xDemoStartSampling(GH3X2X_FUNCTION_ECG)开启；

【3】 **获取 Lead off 事件后关闭 ecg 采样或主控主动关闭 ecg 采样；**获取 lead on 事件是通过 Gh3x2x_LeadOffEventHook 函数获取；ecg 采样时通过 Gh3x2xDemoStopSampling(GH3X2X_FUNCTION_ECG)关闭；

【4】 **主控主动关闭导联检测：**通过调 Gh3x2xDemoStopSampling(GH3X2X_FUNCTION_LEAD_DET)关闭导联检测；

5.2 算法结果获取和处理

请参考《Goodix 算法调用 SDK 接口说明书》第 4 章，算法结果通过 Gh3x2x_EcgAlgorithmResultReport 获得。

Goodix Confidential For pn@lierda.com only

汇顶机密
Goodix Confidential For pn@lierda.com Only

6 Goodix 通信协议移植

GH(M)3x2x 驱动库内置 Goodix 通信协议，用于和 Goodix PC 工具或 Goodix APP 工具连接，用于调试和采集数据。

6.1 移植步骤

6.1.1 开启协议总开关

将 __SUPPORT_PROTOCOL_ANALYZE__ 设置为 1

6.1.2 选择协议类型

将 __PROTOCOL_SERIAL_TYPE__ 设置为__PROTOCOL_SERIAL_USE_UART__ 或 __PROTOCOL_SERIAL_USE_BLE__

宏	可配置值	描述
__PROTOCOL_SERIAL_TYPE__	__PROTOCOL_SERIAL_USE_UART__	串口协议
	__PROTOCOL_SERIAL_USE_BLE__	BLE 协议

6.1.3 调整协议相关宏配置

宏	可配置值	描述
__SUPPORT_ZIP_PROTOCOL__	0	不开启压缩协议
	1	开启压缩协议，会减少蓝牙带宽占用，但会增加更多的 Ram 和 Rom
__GH3X2X_PROTOCOL_SEND_TIMER_PERIOD__	10 到 50（推荐 10）	定时器发送周期（单位：ms），**该时间必须大于 BLE 的连接间隔，否则数据发送可能会阻塞导致数据丢失。**
__GH3X2X_PROTOCOL_DATA_FIFO_LEN__	默认值为 32	协议数据发送 FIFO 深度
__GH3X2X_PROTOCOL_EVENT_FIFO_LEN__	默认值为 16	协议事件发送 FIFO 深度

汇顶科技 机密信息
未经允许 不得转载

__GH3X2X_PROTOCOL_EVENT_WAITING_ACK_TIME__	默认 500	协议事件 ACK 等待时间（单位：ms）
__GH3X2X_PROTOCOL_EVENT_RESEND_NUM__	默认 255	协议事件重发次数（255 表示永久性重发）
__GH3X2X_PROTOCOL_DATA_FUNCTION_INTERCEPT__	默认 0	屏蔽协议上报的功能，默认为 0（全上报） 或： (GH3X2X_FUNCTION_HR|GH3X2X_FUNCTION_HRV) 即表示 HR 与 HRV 的 rawdata 数据等不会随协议上报

6.1.4 BLE 模块设置

1）与 GOODIX APP 对接的 BLE profile 需要按照以下设定：

服务 UUID：”0000190e-0000-1000-8000-00805f9b34fb”

数据 Tx characteristic，连接成功后等待接收 BLE 设备端数据

TX_UUID：”00000003-0000-1000-8000-00805f9b34fb”

数据 Rx characteristic 属性 UUID，连接成功后可发送数据到 BLE 设备端

RX_UUID：”00000004-0000-1000-8000-00805f9b34fb”

2）BLE 改成长包

将 BLE mtu size 设置为 247

6.1.5 下行接口对接（可选）

mcu 接收到 PC/APP 发送的数据后，将数据通过调用 Gh3x2xDemoProtocolProcess 函数传入驱动库

注：如果协议接收处理接口(Gh3x2xDemoProtocolProcess)和驱动库其他操作接口不在同一个线程调用，为了解决资源冲突的问题，请务必在调用驱动库相关接口的地方，加互斥锁保护。

6.1.6 上行接口对接

1）在 mcu 系统中准备一个定时器；

汇顶科技 机密信息
未经允许 不得转载

问题反馈

2）实现 Gh3x2xSerialSendTimerInit 函数（在 gh3x2x_demo_user.c），用于初始化定时器，定时周期和__GH3X2X_PROTOCOL_SEND_TIMER_PERIOD__ 相同；

3）在定时器中断处理中调用 Gh3x2xSerialSendTimerHandle 函数；

4）实现 Gh3x2xSerialSendTimerStart 函数（在 gh3x2x_demo_user.c），用于开启定时器；

5）实现 Gh3x2xSerialSendTimerStop 函数（在 gh3x2x_demo_user.c），用于关闭定时器；

6）实现 Gh3x2x_HalSerialSendData 函数（在 gh3x2x_demo_user.c），用于发送数据；

汇顶机密
Goodix Confidential For pn@lierda.com Only

7 六导联 ECG 功能的额外的移植步骤

由于六导联 ECG 功能是通过两块 GH3x2x 芯片级联实现的，需要额外配置。

7.1 相关宏配置

表 7-1 六导联相关宏配置

宏	可配置值	描述
__GH3X2X_CASCADE_EN__	必须配置为 1	Ecg 级联开关
__GH3X2X_I2C_DEVICE_SLAVER_ID__	GH3X2X_I2C_ID_SEL_1L0L = 0, /**< i2c ID0 & ID1 pin low */ GH3X2X_I2C_ID_SEL_1L0H, /**< i2c ID0 pin high & ID1 pin low */ GH3X2X_I2C_ID_SEL_1H0L, /**< i2c ID0 pin low & ID1 pin high */ GH3X2X_I2C_ID_SEL_1H0H, /**< i2c ID0 & ID1 pin high */	如果使用的是 I2C 接口，则需要再配置从芯片的设备地址
__INTERRUPT_PROCESS_BY_POLLING__	必须配置为 1	目前要使用 6 导联 ecg 方案，数据处理只支持轮询模式

7.2 I2C 接口注册

I2C 注册只需配置__GH3X2X_I2C_DEVICE_ID__(主芯片设备 ID)以及__GH3X2X_I2C_DEVICE_SLAVER_ID__（从芯片设备 ID）

7.3 SPI 接口注册

SPI 接口需要调用接口 GH3x2xGetChipOperateIndexRealtime，来判断当前正在操作主芯片还是从芯片，以下是 spi 部分用户函数的编写伪代码。

```
void hal_gh3x2x_spi_init(void)
{
    /* init spi and cs pin */
    初始化主芯片 spi;
```

```
    初始化从芯片 spi;
}
void hal_gh3x2x_reset_pin_ctrl(GU8 pin_level)
{
    if(0 == GH3x2xGetChipOperateIndexRealtime)
    {
        用户的主芯片 spi cs 操作;
    }
    else
    {
        用户的从芯片 spi cs 操作;
    }
}
```

其他 spi 用户函数的编写类似，这里不再赘述；

相关接口函数介绍见附录一。

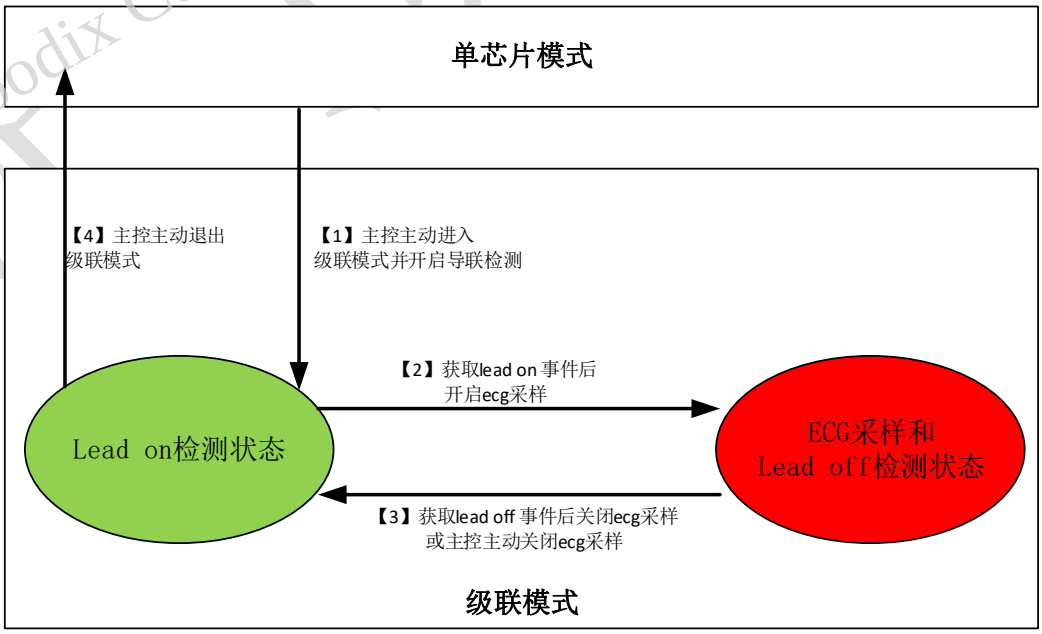
7.4 配置表存放

配置表仍然存在在 gh3x2x_demo_reg_array.c 中，但需要遵循以下规则：

1、主芯片的 ecg 配置表一定要放在 **gh3x2x_reg_list2** 中；
2、从芯片的 ecg 配置表一定要放在 **stGh3x2xEcgCascadeSlaveRealReglist** 中；

7.5 函数的推荐调用流程

控制流程如下图：



图中的各个步骤的具体操作方法描述如下：

【1】 **主控主动进入级联模式并开启导联检测。**通过调用 Gh3x2xEnterCascadeMode();函数后，再调用 Gh3x2xDemoStartSampling(GH3X2X_FUNCTION_LEAD_DET)开启导联检测；

【2】 **获取 lead on 事件后开启 ecg 采样；**获取 lead on 事件是通过 Gh3x2x_LeadOnEventHook 函数获取；ecg 采样时通过 Gh3x2xDemoStartSampling(GH3X2X_FUNCTION_ECG)开启；

【3】 **获取 Lead off 事件后关闭 ecg 采样或主控主动关闭 ecg 采样；**获取 lead on 事件是通过 Gh3x2x_LeadOffEventHook 函数获取；ecg 采样时通过 Gh3x2xDemoStopSampling(GH3X2X_FUNCTION_ECG)开启；

【4】 **主控主动退出级联模式。**通过调用 Gh3x2xExitCascadeMode ()函数退出级联模式。

7.6 算法结果获取和处理

请参考《Goodix 算法调用 SDK 接口说明书》第 4 章，算法结果通过 Gh3x2x_EcgAlgorithmResultReport 获得。

算法结果只输出了 2 路导联结果 ECG_0 和 ECG_1, 6 导联各路数据换算公式如下：

```
ECG_0 = ECG_0;
ECG_1 = ECG_1;
ECG_2 = ECG_0 - ECG_1;
ECG_3 = 0.5*ECG_1 - ECG_0;
ECG_4 = 0.5*ECG_0 - ECG_1;
ECG_5 = 0.5*(ECG_0 + ECG_1);
```

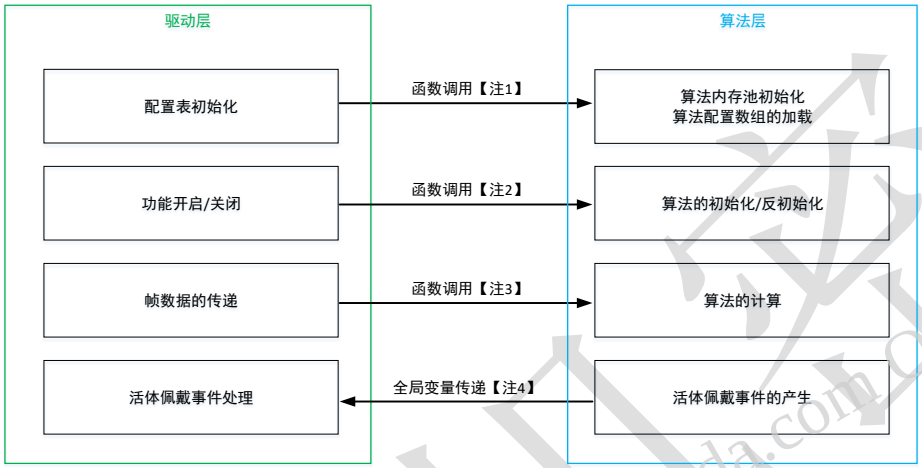
8 驱动和算法分开运行详细修改指引

汇顶机密
Goodix Confidential For pr@lierda.com Only

8.1 驱动和算法交互机制

8.1.1 常规方式的交互机制

下图为常规方式（驱动和算法在同一线程）下驱动和算法交互机制：



注 1：函数调用：通过驱动层的 gh3x2x_init_hook_func 中调用算法层的 GH3X2X_AlgoCallConfigInit 完成；

注 2：函数调用：功能开启：通过驱动层 Gh3x2xDemoStartSampling 中调用算法层 GH3X2X_AlgoInit 完成；功能关闭：通过驱动层 Gh3x2xDemoStopSampling 中调用 GH3X2X_AlgoDeinit 完成；

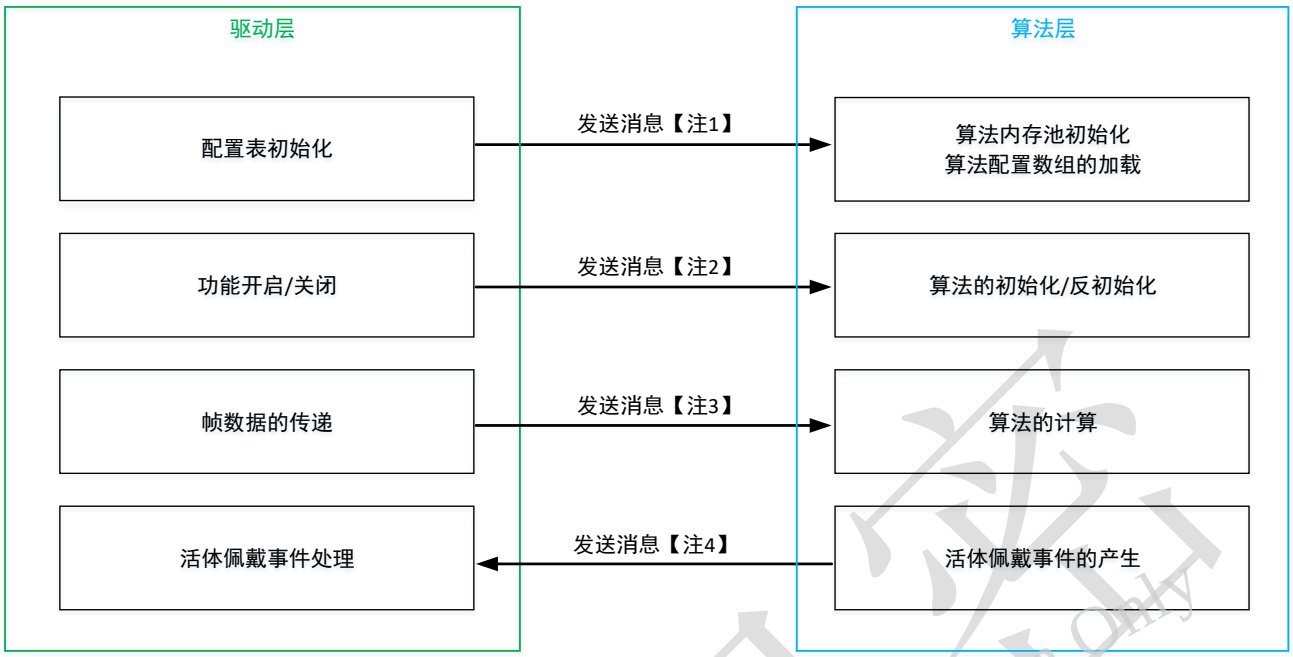
注 3：函数调用：通过驱动层 gh3x2x_algorithm_get_io_data_hook_func 中调用算法层 GH3X2X_AlgoCalculate 完成；

注 4：全局变量传递：通过 GH3X2X_SetSoftEvent 函数的执行会改变驱动库中的全局变量，从而在驱动库中获取该全局变量的信息；

8.1.2 分开运行方式的交互机制

由于驱动和算法分开运行，双方信息传递无法通过函数调用和全局变量传递。因此只能通过消息来传递信息。具体机制如下图：

汇顶机密
Goodix confidential For pn@lierda.com only



注 1：通过在驱动层发送消息（demo 代码已经预留宏定义 GH3X2X_SEND_MSG_ALGO_CFG_INIT）实现；

注 2：通过在驱动层发送消息（demo 代码已经预留宏定义 GH3X2X_SEND_MSG_ALGO_START 和 GH3X2X_SEND_MSG_ALGO_STOP）实现；

注 3：通过在驱动层发送消息（demo 代码已经预留宏定义 GH3X2X_SEND_MSG_ALGO_CAL）实现；

注 4：通过在算法层发送消息（demo 代码已经预留宏定义 GH_SEND_MSG_WEAR_EVENT）实现；

注 3：函数调用：通过驱动层 gh3x2x_algorithm_get_io_data_hook_func 中调用算法层 GH3X2X_AlgoCalculate 完成；

注 4：全局变量传递：通过 GH3X2X_SetSoftEvent 函数的执行会改变驱动库中的全局变量，从而在驱动库中获取该全局变量的信息；

汇顶机密
Goodix Confidential For pn@lierda.com Only

8.2 修改步骤

8.2.1 驱动层的文件修改和编译

1、如果驱动层所在线程或内核仍然会调用部分算法，请保持__DRIVER_LIB_MODE__ 为 __DRV_LIB_WITH_ALGO__；
2、如果驱动层所在线程或内核不会调用任何算法（即全部算法都在另外一个线程或内核），请将 __DRIVER_LIB_MODE__ 设置成__DRV_LIB_WITHOUT_ALGO__；
3、将__GH_MSG_WITH_ALGO_LAYER_EN__ 设置为 1；
4、在 gh3x2x_demo_config.h 中添加 GH3X2X_SEND_MSG_ALGO_CFG_INIT、GH3X2X_SEND_MSG_ALGO_START 等函数中添加消息发送和处理函数；
5、获取到算法层的消息后，需要主动触发调用一次 Gh3x2xDemoInterruptProcess；
6、如果驱动层所在线程或内核仍然会调用部分算法，请将 demo_kernel_code 中的文件 、demo_algo_code 中的文件、驱动库文件、用到的算法库文件和 gh3x2x_drv.h 纳入编译范围；
7、如果驱动层所在线程或内核不会调用任何算法（即全部算法都在另外一个线程或内核），请将 demo_kernel_code 中的文件 、驱动库文件和 gh3x2x_drv.h 纳入编译范围；

8.2.2 算法层的文件修改和编译

1、如果佩戴检测算法和驱动不在同一线程或内核，请将 gh3x2x_demo_algo_config.h 中的 __GH_MSG_WTIH_DRV_LAYER_EN__ 设置为 1；
2、如果佩戴检测算法和驱动不再同一线程或内核，请将 gh3x2x_demo_algo_config.h 中的 __GH_MSG_WTIH_DRV_LAYER_EN__设置为 0；
3、将 gh3x2x_demo_algo_config.h 中__GH_SEPARATE_ALGO_EN__设置为 1
4、在 gh3x2x_demo_algo_config.h 中添加 GH3X2X_SEND_MSG_ALGO_CFG_INIT、GH3X2X_SEND_MSG_ALGO_START 等函数中添加消息发送；
5、请将 demo_algo_code 中的文件 、算法库文件、gh3x2x_demo_config.h 和 gh3x2x_drv.h 纳入编译范围；
6、获取到驱动层的消息后，做对应处理（**注意：在调用算法接口前，需要对 g_pstGh3x2xFrameInfo 进行数据填充，填充说明见小节” g_pstGh3x2xFrameInfo 的填充”**），伪代码如下：

汇顶科技 机密信息
未经允许 不得转载

问题反馈

```
while(1)
{
  //if(receive MSG_ALGO_CFG_INIT)
  GH3X2X_AlgoCallConfigInit(g_pstGh3x2xFrameInfo,XXX);  //XXX comes from msg

  //if(receive MSG_ALGO_START)
  GH3X2X_AlgoInit(GH3X2X_FUNCTION_XXX);  //GH3X2X_FUNCTION_XXX comes from msg, for example GH3X2X_FUNCTION_HR

  //if(receive MSG_ALGO_STOP)
  GH3X2X_AlgoDeinit(GH3X2X_FUNCTION_XXX);  ///GH3X2X_FUNCTION_XXX comes from msg, for example GH3X2X_FUNCTION_HR

  //if(receive MSG_ALGO_CAL)
  GH3X2X_AlgoCalculate(GH3X2X_FUNCTION_XXX);  //GH3X2X_FUNCTION_XXX comes from msg, for example GH3X2X_FUNCTION_HR


}
```

8.2.3 g_pstGh3x2xFrameInfo 的填充

1、GH3X2X_AlgoCallConfigInit 执行前，g_pstGh3x2xFrameInfo 无需填充；

2、GH3X2X_AlgoInit 执行前，需填充项如下：

填充变量	描述
pstFrameInfo ->pstFunctionInfo ->usSampleRate	采样率

3、GH3X2X_AlgoCalculate 执行前，需填充项如下：

填充变量	描述
pstFrameInfo->punFrameRawdata	各通道 rawdata
pstFrameInfo->pusFrameGsensordata	Gsensor 数据
pstFrameInfo->punFrameAgcInfo	调光信息（驱动电流、增益）
pstFrameInfo->punFrameFlag[0]	调光标记
pstFrameInfo->punFrameCnt	帧计数

4、GH3X2X_AlgoDeinit 执行前，g_pstGh3x2xFrameInfo 无需填充；

9 附录一：Demo 与驱动库函数查询表

表 9-1 gh3x2x_demo.h 函数

函数名	Gh3x2xDemoInit
描述	GH(M)3x2x 芯片总初始化
入参	无
出参	无

返回值	GH3X2X_RET_OK 初始化成功 GH3X2X_RET_COMM_ERROR GH3X2X 芯片初始化未成功（通讯失败）	
函数名	**Gh3x2xDemoInterruptProcess**	
描述	GH(M)3x2x 芯片中断处理函数。内部包含对各种中断事件的处理	
入参	无	
出参	无	
返回值	无	
函数名	**Gh3x2xDemoApplicationModeSwitch**	
描述	开启功能采样	
入参	GU32 unFuncMode	需要开启采样的功能 GH3X2X_FUNCTION_ADT：ADT 采样 GH3X2X_FUNCTION_HR：HR 采样 GH3X2X_FUNCTION_SPO2：SPO2 采样 GH3X2X_FUNCTION_ECG：ECG 采样
出参	无	
返回值	无	
函数名	**Gh3x2xDemoStartSampling**	
描述	开启功能采样	
入参	GU32 unFuncMode	需要开启采样的功能 GH3X2X_FUNCTION_ADT：ADT 采样 GH3X2X_FUNCTION_HR：HR 采样 GH3X2X_FUNCTION_SPO2：SPO2 采样 GH3X2X_FUNCTION_ECG：ECG 采样
出参	无	
返回值	无	
函数名	**Gh3x2xDemoStopSampling**	
描述	关闭功能采样	
入参	GU32 unFuncMode	需要开启采样的功能 GH3X2X_FUNCTION_ADT：ADT 采样 GH3X2X_FUNCTION_HR：HR 采样 GH3X2X_FUNCTION_SPO2：SPO2 采样 GH3X2X_FUNCTION_ECG：ECG 采样
出参	无	
返回值	无	
函数名	**Gh3x2xCreateAdtConfirmTimer**	
描述	创建佩戴检测定时器	
入参	无	
出参	无	
返回值	无	
函数名	**Gh3x2xDemoMoveDetectTimerHandler**	
描述	获取 Gsensor 数据的定时器中断处理函数	

汇顶科技 机密信息
未经允许 不得转载

问题反馈

入参	无	
出参	无	
返回值	无	
函数名	**Gh3x2xDemoProtocolProcess**	
描述	驱动库协议处理函数，当接收到上位机下发的协议数据时，可调用该函数对协议进行解析和回复。	
入参	GU8* puchProtocolDataBuffer	需要驱动库解析的协议数据
	GU16 usRecvLen	协议数据具有的 byte 长度
出参	无	
返回值	无	

表 9-2 snpintf log 函数

函数名	**GH3X2X_Log**	
描述	log 打印函数	
入参	GCHAR *log_string	已经格式化的字符串，只需用类似 printf 函数打印 如：printf("%s",log_string)
出参	无	
返回值	无	

表 9-3 printf log 函数

函数名	**GH3X2X_RegisterPrintf**	
描述	log 打印函数	
入参	pPrintfUser	指向函数的指针的指针 用户需要此指针的实体赋值。 如：(*pPrintfUser) = printf
出参	无	
返回值	无	

表 9-4 I2C 操作函数

函数名	**hal_gh3x2x_i2c_init**	
描述	I2C 接口初始化	
入参	无	
出参	无	
返回值	无	
函数名	**hal_gh3x2x_i2c_write**	
描述	I2C 写数据接口	
入参	GU8 device_id	i2c 地址
	const GU8 write_buffer[]	数据 buffer
	GU16 length	写数据长度（单位：byte）
出参	无	
返回值	错误码 0：成功	

Goodix Confidential For Anafield.com only

函数名	hal_gh3x2x_i2c_read	
描述	I2C 写数据接口	
入参	GU8 device_id	i2c 地址
	const GU8 write_buffer[]	写数据 buffer
	GU16 write_length	写数据长度（单位：byte）
	GU8 read_buffer[]	读数据 buffer
	GU16 read_length	读数据长度（单位：byte）
出参	无	
返回值	错误码 0：成功	

表 9-5 SPI 操作函数

函数名	hal_gh3x2x_spi_init	
描述	SPI 接口初始化	
入参	无	
出参	无	
返回值	无	
函数名	hal_gh3x2x_spi_write	
描述	SPI 写数据接口（注意：该函数接口中，须确保不能包含 SPI 总线 CS 管脚的控制，该管脚仅受驱动库控制。）	
入参	const GU8 write_buffer[]	数据 buffer
	GU16 length	写数据长度（单位：byte）
出参	无	
返回值	错误码 0：成功	
函数名	hal_gh3x2x_spi_read	
描述	SPI 读数据接口（注意：该函数接口中，用户无需实现写数据功能，且须确保不能包含 SPI 总线 CS 管脚的控制，该管脚仅受驱动库控制。）	
入参	GU8 read_buffer[]	读数据 buffer
	GU16 length	读数据长度（单位：byte）
出参	无	
返回值	错误码 0：成功	
函数名	hal_gh3x2x_spi_cs_ctrl	
描述	SPI CS 引脚操作接口（软件 CS 使能时需实现该函数）	
入参	GU8 cs_pin_level	cs 引脚高低电平设置
出参	无	
返回值	无	
函数名	hal_gh3x2x_spi_write_F1_and_read	
描述	SPI 读数据接口，CS pin 由 SPI 硬件模块自动控制，该函数需实现 CS LOW + F1(W) + read_buf[0](R) + read_buf[1](R) + ... + CS HIGH 时序	
入参	GU8 read_buffer[]	读数据 buffer
	GU16 length	读数据长度（单位：byte）
出参	无	

汇顶科技机密信息 Goodix Confidential For Product Evaluation Only

汇顶科技 机密信息
未经允许 不得转载

返回值	错误码 0：成功

表 9-6 reset 操作函数

函数名	hal_gh3x2x_reset_pin_init	
描述	硬件复位管脚初始化	
入参	无	
出参	无	
返回值	无	
函数名	**hal_gh3x2x_reset_pin_ctrl**	
描述	复位管脚电平控制函数	
入参	GU8 pin_level	reset 引脚高低电平设置
出参	无	
返回值	无	

表 9-7 INT 操作函数

函数名	hal_gh3x2x_int_init
描述	中断管脚初始化
入参	无
出参	无
返回值	无

表 9-8 延时操作函数

函数名	Gh3x2x_BspDelayUs	
描述	微秒延时函数	
入参	GU16 usUsec	微秒
出参	无	
返回值	无	
函数名	**Gh3x2x_BspDelayMs**	
描述	毫秒延时函数	
入参	GU16 usMsec	毫秒
出参	无	
返回值	无	

表 9-9 电流处理函数

函数名	Gh3x2x_UserHandleCurrentInfo
描述	用户处理电流信息
入参	无
出参	无
返回值	无

表 9-10 内存处理函数

函数名	Gh3x2xMallocUser

汇顶科技 机密信息 未经允许 不得转载

问题反馈

描述	复位管脚电平控制函数	
入参	GU32 unMemSize	内存池大小
出参	无	
返回值	内存地址	
函数名	**Gh3x2xFreeUser**	
描述	复位管脚电平控制函数	
入参	void* MemAddr	内存池地址
出参	无	
返回值	无	

表 9-11 其它用户代码函数

功能模块	相关用户函数
佩戴检测	Gh3x2x_CreateAdtConfirmTimer Gh3x2x_StartAdtConfirmTimer Gh3x2x_StopAdtConfirmTimer
通信协议	Gh3x2x_HalSerialSendData Gh3x2xSerialSendTimerInit Gh3x2xSerialSendTimerStop Gh3x2xSerialSendTimerStart

表 9-12 gsensor 操作函数

函数名	**hal_gsensor_start_cache_data**	
描述	开始缓存 G-sensor 数据	
入参	无	
出参	无	
返回值	内存地址	
函数名	**hal_gsensor_stop_cache_data**	
描述	停止缓存 G-sensor 数据	
入参	无	
出参	无	
返回值	无	
函数名	**hal_gsensor_drv_get_fifo_data**	
描述	抓取 gsensor fifo 中的数据	

入参	STGsensorRawdata gsensor_buffer[]	G-sensor 数据 buffer
		typedef struct
		{
		GS16 sXAxisVal; // X-Axis Value
		GS16 sYAxisVal; // Y-Axis Value
		GS16 sZAxisVal; // Z-Axis Value
		#if _GS_GYRO_ENABLE_
		GS16 sXGyroVal;

		GS16 sYGyroVal; GS16 sZGyroVal; #endif }STGsensorRawdata;
	GU16 *gsensor_buffer_index	G-sensor 数据个数，每个 G-sensor 数据包含 x,y,z 轴的采样数据
出参	无	
返回值	无	

表 9-13 fifo 水线操作函数

函数名	GH3X2X_FifoWatermarkThrConfig	
描述	设置 Fifo 水线函数（注意事项见：“采样控制”的“Fifo 中断水线设定”章节）	
入参	GU16 usFifoWatermarkThr	Fifo 水线设定值 范围为(2, 800]
出参	无	
返回值	错误码	

表 9-14 中断模式操作函数

函数名	Gh3x2xInterruptModeSwitch	
描述	设置中断模式函数（__INTERRUPT_PROCESS_BY_POLLING__ == (__POLLING_INT_PROCESS_MODE__)有效）	
入参	GU8 uchIntModeType	0：中断响应模式 1：轮询模式
出参	无	
返回值	无	
函数名	**Gh3x2xGetInterruptMode**	
描述	获取当前中断模式函数	
入参	无	
出参	无	
返回值	中断模式	

表 9-15 采样类 hook 函数

函数名	gh3x2x_init_hook_func
描述	初始化 hook 函数
入参	无
出参	无
返回值	无
函数名	**gh3x2x_sampling_start_hook_func**
描述	开始采样 hook 函数
入参	无
出参	无
返回值	无
函数名	**gh3x2x_sampling_stop_hook_func**
描述	停止采样 hook 函数

Goodix Confidential For Aiqiyuda.com Only

入参	无	
出参	无	
返回值	无	
函数名	**gh3x2x_get_rawdata_hook_func**	
描述	读取 FIFO 数据 hook 函数	
入参	无	
出参	GU16 length	buffer 长度
	GU8 *read_buffer_ptr	FIFO 数据 buffer 地址
返回值	无	

表 9-16 算法输入输出信息获取 hook 函数

函数名	**gh3x2x_algorithm_get_io_data_hook_func**	
描述	算法输入输出信息获取 hook 函数	
入参	无	
出参	const STGh3x2xFrameInfo * const pstFrameInfo	可获取的信息的描述在如下结构体中（该结构体定义在 gh3x2x_drv.h）： 结构体参考附录二
返回值	无	

表 9-17 功能类 hook 函数

函数名	**gh3x2x_reset_by_protocol_hook**	
描述	由 BLE 协议进行复位的 hook 函数	
入参	无	
出参	无	
返回值	无	
函数名	**gh3x2x_config_set_start_hook**	
描述	开始加载虚拟寄存器部分配置 hook 函数	
入参	无	
出参	无	
返回值	无	
函数名	**gh3x2x_config_set_stop_hook**	
描述	结束加载虚拟寄存器部分配置 hook 函数	
入参	无	
出参	无	
返回值	无	
函数名	**gh3x2x_write_algo_config_hook**	
描述	加载算法用配置 hook 函数	
入参	无	
出参	GU16 usVirtualRegAddr	配置地址
	GU16 usVirtualRegValue	配置值
返回值	无	

汇顶科技机密信息 Confidential Information of Goodix Technology

表 9-18 唤醒/睡眠操作函数

函数名	Gh3x2x_BeforeWakeUpHook
描述	中断函数唤醒 hook 函数
入参	无
出参	无
返回值	无
函数名	**Gh3x2x_WakeUpHook**
描述	中断函数睡眠 hook 函数
入参	无
出参	无
返回值	无

表 9-19 采样类 hook 函数

函数名	Gh3x2x_LeadOnEventHook	
描述	lead on hook 函数	
入参	无	
出参	无	
返回值	无	
函数名	**Gh3x2x_LeadOffEventHook**	
描述	lead off hook 函数	
入参	无	
出参	无	
返回值	无	
函数名	**Gh3x2x_WearEventHook**	
描述	wear event hook 函数	
入参	无	
出参	GU16 usGotEvent	中断事件
	GU8 uchExentEx	0：硬件脱落 1：非活体脱落
返回值	无	

表 9-20 工程模式启动 hook 函数

函数名	Gh3x2xDemoStartSamplingForEngineeringMode	
描述	工程模式开启功能 函数	
入参	GU32 unFuncMode	需要启动的功能（使用方法同 Gh3x2xDemoStartSampling）
	GU8 uchArrayCfgIndex	需要设置的配置 ID（默认为 0）
	STGh3x2xEngineeringModeSampleParam *pstSampleParaGroup	采样参数结构体数组的首地址指针
	GU8 uchSampleParaGroupNum	采样参数组结构体数组的长度
出参	无	

汇顶科技 机密信息
未经允许 不得转载

问题反馈

返回值	无

表 9-21 工程模式停止 hook 函数

函数名	Gh3x2xDemoStopSamplingForEngineeringMode
描述	工程模式停止功能 函数
入参	无
出参	无
返回值	无

表 9-22 六导联相关函数

函数名	GH3x2xGetChipOperateIndexRealtime
描述	获取当前驱动库正在操作芯片索引，该函数指在用户 spi 函数中会用到；
入参	无
出参	无
返回值	0：主芯片 1；从芯片
函数名	**Gh3x2xEnterCascadeMode**
描述	进入级联模式。该函数被调用时，驱动库已开启的功能会被全部关闭；
入参	无
出参	无
返回值	无
函数名	**Gh3x2xExitCascadeMode**
描述	退出级联模式。该函数被调用时，驱动库已开启的功能会被全部关闭；
入参	无
出参	无
返回值	无

Goodix Confidential For pm@lierda.com Only

10 附录二：STGh3x2xFrameInfo 的定义

STGh3x2xFrameInfo 成员描述

成员名	类型	描述
unFunctionID	GU32	功能 ID 号：如 GH3X2X_FUNCTION_HR
pstFunctionInfo	STGh3x2xFunctionInfo	功能信息：详见下表 STGh3x2xFunctionInfo 的成员描述
pstDownSampleInfo	STGh3x2xDownSampleInfo	降采样相关信息：详见下表 STGh3x2xDownSampleInfo 的成员描述
uchFuntionChnlLimit	GU8	该功能最多支持的通道数
pchChnlMap	GU8 *	指向通道映射表的指针
punFrameRawdata	GU32 *	指向 Rawdata 数组的指针，每个 Rawdata 占用 32bit；
pusFrameGsensordata	GS16 *	指向 Gsensor 数组的指针； ACCX: pusFrameGsensordata[0] ACCY: pusFrameGsensordata[1] ACCZ: pusFrameGsensordata[2]
pstFrameCapdata	STCapRawdata	电容数据 GU32 unCapData[4];
pstFrameTempdata	STTempRawdata	温度 sensor 数据 GU32 unTempData[4];
punFrameAgcInfo	GU32 *	指向调光信息数组的指针 每个通道的调光信息占用 32Bit 含义如下： [3:0] Gain [5:4] Bg cancel [7:6] Res [15:8] Drv0 电流 [23:16] Drv1 电流 [31:24] Res

Goodix Confidential For Reference Only

punFrameFlag	GU32 *	一些可能用到的标记 punFrameFlag[0] bit0: Channel0 是否在调光 0:否 1:是 bit1: Channel1 是否在调光 0:否 1:是 bitN: ChannelN 是否在调光 0:否 1:是 punFrameFlag[1] bit0: Channel0 调光方向 0: 向上 1:向下 bit1: Channel1 调光方向 0: 向上 1:向下 bitN:ChannelN 调光方向 0: … punFrameFlag[2] bit0：ECG 是否处于快速恢复状态 0：不是 1：是 bit1: 是否是新的一段数据的起点
punIncompleteRawdata	GU32 *	FIFO 数据处理过程的中间数据，客户可忽略
puchFrameLastGain	GU8 *	FIFO 数据处理过程的中间数据，客户可忽略
punFrameCnt	GU32 *	帧计数器
pstAlgoResult	STGh3x2xAlgoResult *	算法结果
pstAlgoRecordResult	STGh3x2xAlgoResult *	部分算法结果记录，供其它模块使用

STGh3x2xFunctionInfo

成员名	类型	描述
usSampleRate	GU16	该功能的采样率
uchChnlNum	GU8	该功能的通道数
uchSlotBit	GU8	该功能对应的 SlotBit

STGh3x2xDownSampleInfo

成员名	类型	描述
uchDownSampleFactor	GU8	0：降采样不执行 N：降采样比例为：N+1:1
uchDownSampleCnt	GU8	记录用，客户可忽略

STGh3x2xAlgoResult

成员名	类型	描述
uchUpdateFlag	GU8	0：无结果输出 1：有结果输出
uchResultNum	GU8	结果数量
snResult	GS32	snResult[16]算法结果 buffer

汇顶科技 机密信息
未经允许 不得转载

问题反馈

11 附录三：STGh3x2xEngineeringModeSampleParam 定义

工程模式输入结构体表

结构体成员名	描述	设定范围说明
uchFuncitonId	功能 ID	可设置值 GH3X2X_FUNCTION_ADT GH3X2X_FUNCTION_HR GH3X2X_FUNCTION_HRV GH3X2X_FUNCTION_SPO2
uchSampleRateChangeEn	是否需要改变采样率	0：不改变　1：改变
usSampleRate	积分时间（Hz）	范围：5Hz~1000Hz 且**必须等于 1000 除以 X** (X 为≥1 的整数) 举例： 合法值：5、1000、500、125、1000/3 非法值：0、、1001、128 另外，多功能同时运行，采样率有如下制约关系 如最快功能的是 100Hz（换算成周期是 10ms），那么其他的功能的采样周期必须是 10ms 的倍数（10ms、20 ms、30ms、40ms、50ms...）,那么对应的可以配置的频率是 100Hz、50Hz、33.3Hz、25Hz、20Hz...
uchIntTimeChangeEn	是否需要改变积分时间	0：不改变　1：改变
uchIntTime	积分时间	GH3X2X_ENGINEERING_MODE_INT_TIME_10_US GH3X2X_ENGINEERING_MODE_INT_TIME_20_US GH3X2X_ENGINEERING_MODE_INT_TIME_30_US GH3X2X_ENGINEERING_MODE_INT_TIME_39_US GH3X2X_ENGINEERING_MODE_INT_TIME_79_US GH3X2X_ENGINEERING_MODE_INT_TIME_158_US GH3X2X_ENGINEERING_MODE_INT_TIME_316_US
uchTiaGainChangeEn	是否需要调整 TIA Gain	0：不改变　1：改变

汇顶机密
Goodix Confidential For Reference Only

uchTiaGain[X]	第 X 通道的 TIA Gain	GH3X2X_ENGINEERING_MODE_TIA_GAIN_10_K GH3X2X_ENGINEERING_MODE_TIA_GAIN_25_K GH3X2X_ENGINEERING_MODE_TIA_GAIN_50_K GH3X2X_ENGINEERING_MODE_TIA_GAIN_75_K GH3X2X_ENGINEERING_MODE_TIA_GAIN_100_K GH3X2X_ENGINEERING_MODE_TIA_GAIN_250_K GH3X2X_ENGINEERING_MODE_TIA_GAIN_500_K GH3X2X_ENGINEERING_MODE_TIA_GAIN_750_K GH3X2X_ENGINEERING_MODE_TIA_GAIN_1000_K GH3X2X_ENGINEERING_MODE_TIA_GAIN_1250_K GH3X2X_ENGINEERING_MODE_TIA_GAIN_1500_K GH3X2X_ENGINEERING_MODE_TIA_GAIN_1750_K GH3X2X_ENGINEERING_MODE_TIA_GAIN_2000_K
chLedCurrentChangeEn	是否需要调整 LED 电流	0：不改变　1：改变
chLedDrv0Current[X]	第 X 通道的 Driver0 LED 电流（mA）	0~200mA，步进为 1mA 注意：如果存在两个通道共用一个 slot，那么最终设定的电流以索引值大的通道为准；
chLedDrv1Current[X]	第 X 通道的 Driver1 LED 电流（mA）	0~200mA，步进为 1mA 注意：如果存在两个通道共用一个 slot，那么最终设定的电流以索引值大的通道为准；

Goodix Confidential For Goodix Only

12 附录四：移植过程常见问题排查

12.1 编译出错

现象	排查方向	子步骤
关闭或打开部分宏后编译报错	检查是否是标准 C 函数（如 malloc、free、snprintf）当前平台是否未支持	1、如果 malloc、free 未支持，请换成平台支持的等效函数 2、如果是 snprintf 未支持，请使用 __EXAMPLE_LOG_METHOD_1__ 打 log

12.2 链接出错

现象	排查方向	子步骤
驱动库占用 Ram 或 Rom 超出预期	部分配置宏可能没有关掉	检查每一个 config.h 中宏，将当前项目用不到的功能关掉，合理设置 buffer 大小
关闭或打开部分宏后链接报错：驱动或 demo code 中部分函数找不到实体	是否因链接顺序报错	尝试采样静态库组的方式链接，如 8.2.a, cmake 工具下的语法参考
关闭或打开部分宏后链接报错：__sbrk，__write, __close, __fstat, _isatty, __lseek, _read 找不到实体	当前平台可能不支持 snprintf	Log 方式采用 __EXAMPLE_LOG_METHOD_1__
	可能将未用到的函数链接进去了	排查链接选项是否有问题，如 cmake 工具下 —gc-section 选项是否有添加

例 8.2.a　cmake 工具下的语法参考

```
target_link_libraries(target_name
-Wl,--start-group
libgh3x2x_drv_cortexM4l_band_gcc_hard_agc.a
libgh3x2x_drv_cortexM4l_band_gcc_hard_algo_call.a
libgh3x2x_drv_cortexM4l_band_gcc_hard_ecg.a
libgh3x2x_drv_cortexM4l_band_gcc_hard_kernel.a
libgh3x2x_drv_cortexM4l_band_gcc_hard_soft_adt.a
libgh3x2x_drv_cortexM4l_band_gcc_hard_zip.a
-Wl,--end-group
)
```

汇顶科技 机密信息
未经允许 不得转载

问题反馈

12.3 运行异常

现象	排查方向	子步骤
程序卡在某个函数出不来	是否掉入 while(1)陷阱中	1、关闭 mcu 看门狗 2、打开 log，看 log 是否有报错 3、单步
驱动库输出以下 log：“Gh3x2xDemoInit:init fail, error code: -4”	通讯不通	1、检查硬件连接、电源是否正常 2、用逻辑分析仪分析时序和 Datasheet 上的做比对，确认时序是否存在异常
概率性开启功能失效或数据异常	顶层接口是否有做互斥处理	“Gh3x2xDemoStartSampling”、“Gh3x2xDemoStopSampling”、“Gh3x2xDemoProtocolProcess”、“Gh3x2xDemoInit”、“Gh3x2xDemoInterruptProcess”以及“Gh3x2xDemoMoveDetectTimerHandler”如果没有放在同一个线程，需要加锁防止并行冲突。
	Stack 不够用	确认当前工作模式下，Stack 设定是否满足当前版本资源统计表中的 stack 要求。如不满足，请增大。

Goodix Confidential For Internal Use Only

13 附录四：I2C 设备地址宏配置与 pin 脚连接关系

宏	MISO 状态	CS 状态	设备地址（7bit）	写地址（8bit）	读地址（8bit）
GH3X2X_I2C_ID_SEL_1L0L	LOW	LOW	0x14	0x28	0x29
GH3X2X_I2C_ID_SEL_1L0H	LOW	HIGH	0x15	0x2A	0x2B
GH3X2X_I2C_ID_SEL_1H0L	HIGH	LOW	0x16	0x2C	0x2D
GH3X2X_I2C_ID_SEL_1H0H	HIGH	HIGH	0x17	0x2E	0x2F

汇顶机密
Goodix Confidential For pn@lierda.com Only

14 版本记录

表 14-1 版本记录

文件版本	修改时间	修订
V1.0	2020-05-11	正式发布版
V1.1	2020-08-14	提供新的示例代码，重新编写文档
V1.2	2020-11-15	1、修改了部分描述便于理解； 2、增加软件定时器相关接口说明； 3、增加部分移植注意事项说明
V1.3	2020-12-04	针对新版驱动库和 demo code 1、整理了 configure 中的 宏分类和描述 2、增加不同模式下外层调用的设计章节（是否带操作系统以及是否是轮询模式） 3、增加 Gh3x2x_UserHandleCurrentInfo 的接口描述；
V1.4	2020-12-16	增加 __SUPPORT_ALGO_INPUT_OUTPUT_DATA_HOOK_CONFIG__ 宏的描述以及对应的 Hook 函数 gh3x2x_algorithm_get_io_data_hook_func 的使用方法
V1.5	2020-12-11	增加__SUPPORT_ENGINEERING_MODE__的描述，以及其对于的工程模式相关的接口
V1.5.1	2021-01-07	增加工程模式参数设定范围/档位说明
V2.0	2021-02-08	更新与修改相关信息，适配新版驱动库
V2.1	2021-02-08	更新算法 API 与 Hook 相关内容
V2.2	2021-02-09	更新 config 与 ResultReport 函数介绍
V2.3	2021-04-09	修改通信协议移植章节 优化文档结果，增加附录
V2.4	2021-04-13	去除__GH3X2X_MULTI_APLICATION_MODE_ENABLE__描述，增加__GH3X2X_ARRAY_CFG_MANUAL_SWITCH_EN__的描述

汇顶科技 机密信息
未经允许 不得转载

问题反馈

V2.5	2021-04-14	Goodix 通信协议移植->上行接口对接 增加调用 Gh3x2xSerialSendTimerHandle 的描述
V2.6	2021-04-15	应用模式切换章节描述修改
V2.7	2021-07-16	更新佩戴检测内容、硬件 SPI 操作接口、G-sensor 增加 gyro 成员，心率结果更新、STGh3x2xFrameInfo 的 时间戳改成帧计数器、配置宏文件分增加描述，增加调光相关宏配置
V2.8	2021-08-30	补完缺失内容，驱动版本对应 v3.2.4.2
V2.9	2021-12-08	补充部分功能宏说明，补充血氧结果说明
V3.0	2021-12-09	更改 Gh3x2xDemoinit 函数调用描述
V3.1	2021-12-18	1) 血氧结果说明 2) BSP 宏说明错误更改
V3.2	2021-12-20	1、补充级联应用相关宏配置 2、补充动态内存相关配置以及用户代码 3、补充 LOG 相关配置以及用户代码 4、在软件运行环境中高亮显示不支持动态链接库。 5、增加附录：移植过程常见问题排查 6、增加：6 导联 ECG 功能的额外的移植步骤 7、增加附录：I2C 设备地址宏配置与 pin 脚连接关系 8、修改__GH3X2X_PROTOCOL_SEND_TIMER_PERIOD__描述：小于 BLE 连接间隔时间改成大于 BLE 连接间隔时间 9、增加运行异常：概率性开启功能失效或数据异常的排查方向
V3.3	2021-12-21	1) 增加 Fifo 水线修改函数说明 2) 增加中断模式切换使用说明 3) 增加运行异常：stack 大小的排查
V3.4	2021-12-29	1) 补充 Demo Init 初始化失败的 error code 说明 2) 增加算法内存模块说明 3) 补充心率血氧通道数宏配置说明 4) 增加对应驱动版本号记录

汇顶机密
Goodix Confidential For Jie@jierda.com Only

V3.5	2022-01-17	1) 更改活体检测部分说明 2) 修改 wearhook 函数说明 3) 增加示例活体检测流程 4) 增加活体检测算法 Report 函数说明
V4.0	2022-02-16	1) 说明函数说明格式 2) 删除算法相关 3) 根据 V4 驱动库版本增加部分函数说明与宏说明 4) 更新活体检测流程
V4.1	2022-02-18	更新文件结构
V4.2	2022-04-27	更改部分 FAE 提出 bug
V4.4	2020-10-20	1、删除了驱动库功能简介中对于移植无关的介绍 2、增加了驱动库文件介绍 3、修改了“添加配置表”章节内容，新增“算法配置表的添加”章节 4、增加了“驱动和算法分开运行详细修改指引”章节； 5、增加了“用软件来代替硬件检测模块”章节； 6、增加了“ECG 功能使用说明”章节； 7、增加 GH3X2X_FifoWatermarkThrConfig 使用方法的说明； 8、增加“软件佩戴检测流程图说明”章节； 9、增加常见问题排查中增加驱动库输出以下 log： “Gh3x2xDemoInit:init fail, error code: -4”的排查操作步骤

Goodix Confidential For Hieraccom Only